

# Worst-Case Optimal BGPs on Temporal Graphs

Diego Arroyuelo  
PUC Chile & IMFD, Chile

Gonzalo Navarro  
U. de Chile & IMFD, Chile

Aidan Hogan  
U. de Chile & IMFD, Chile

Juan Reutter  
PUC Chile & IMFD, Chile

## ABSTRACT

We study how to evaluate basic graph patterns (BGPs) over *temporal* labeled graphs—where each edge is associated with an interval of temporal validity—in a worst-case-optimal manner. We adopt a flexible query language in which users specify  $m$  quads of the form (subject, property, object, time), using constants or variables. The time component denotes the instant at which a particular edge is valid, and users may also include order relations between temporal constants or variables. The answer to such extended BGPs is the set of all valid variable assignments, including those that involve time. We describe an index structure that, for a temporal graph with  $N$  edges, requires  $O(N)$  space and can evaluate extended BGPs in worst-case optimal time  $O(Q^* m \log N)$ , where  $Q^*$  represents the maximum number of solutions for query  $Q$  over any temporal graph with the same number of instants of edge validities. We use our index to adapt Leapfrog Triejoin to the temporal graph setting and can therefore consider any variable evaluation ordering. Moreover, the same index yields worst-case-optimal guarantees for related query types, including snapshot evaluation, version queries, and other temporal variants. Experiments on real-world datasets show that our approach answers realistic queries in milliseconds with low space overhead.

### PVLDB Reference Format:

Diego Arroyuelo, Aidan Hogan, Gonzalo Navarro, and Juan Reutter.  
Worst-Case Optimal BGPs on Temporal Graphs. PVLDB, 14(1): XXX-XXX, 2020.  
doi:XX.XX/XXX.XX

### PVLDB Artifact Availability:

The source code, data, extended version and/or other artifacts have been made available at <https://github.com/darroyue/bgps-temporal-graphs>

## 1 INTRODUCTION

A significant advance on the problem of querying graph databases, in particular for solving Basic Graph Patterns (BGPs), was the discovery of the AGM bound [7] and the consequent definition and development of worst-case-optimal (wco) algorithms. A graph database, in the simplest RDF-like format, is seen as a set of triples  $(s, p, o)$  for subject, predicate, and object. A BGP is a set of triple-patterns  $(s, p, o)$ , where each component can be a constant or a variable. Solving a BGP consists of finding all the assignments of

values to the variables so that the set of triple-patterns, instantiated to triples, appears in the graph. A wco algorithm takes time proportional to the size of the output of the query on some graph. There are already various wco algorithms; the simplest and most popular one is probably Leapfrog TrieJoin (LTJ) [35], which binds—i.e., finds all the possible values of—one variable, and for each value assigned to the variable recurses on the remaining variables.

In this paper we extend this technology to *temporal graphs*, where triples become quads  $(s, p, o, [ti, tf))$ , meaning that triple  $(s, p, o)$  in the graph was valid from time  $ti$  (inclusive) to time  $tf$  (exclusive). Temporal graphs are used when relations have a temporal range of validity (e.g., when a researcher was affiliated with a university), or a temporal instant (e.g., when a conference took place), and allow for querying complex relations (i.e., subgraphs, paths) taking into account those time ranges and instants (e.g., which researchers were affiliated with a university when a given conference was held).

We aim at solving BGPs on temporal graphs. The simplest form of such BGP queries is the *return all time points* query, which looks to find all answers for the BGP that were valid at some point in time, together with the time where each of these answers exists in the database. A folklore way to solve this problem is a so-called *join-first* strategy: we first compute the answers of the BGP disregarding the time information, and then, for each solution to the BGP, we use the time information of the concrete edges to determine the points in time this solution is valid, if any. Hu et al. [21] introduce a so-called *time-first* strategy, which sweeps over the time component and recomputes the BGP each time an event changes the set of edges present in the graph. Though they show good theoretical results for restricted kinds of BGPs, in the general case the whole BGP is recomputed for every event.

We argue that these solutions suffer from two main limitations. First, on the usability side, the *return all time points* query may lack expressiveness in several applications. For example, the latest SQL 2011 standard recognizes the importance of returning answers that mix temporal constraints (e.g., joining two triples on the condition that one precedes a certain timestamp and the other comes after it) [25]. Second, on the algorithmic side, both time- and join-first algorithms translate to fixing particular variable binding orderings on LTJ or on any wco algorithm of choice. But it is well known that choosing a good variable binding ordering is crucial for efficient query resolution, even if all orderings are theoretically optimal [35]. Hence, systems processing temporal graph queries would benefit significantly from an approach that would allow for choosing any variable binding order when using a wco algorithm.

We introduce a linear-space temporal graph representation that overcomes both limitations. Our index structure stores temporal graphs in terms of intervals defined by their updates, using an improvement of versioned binary trees that avoids a space blowup

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing [info@vldb.org](mailto:info@vldb.org). Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.  
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.  
doi:XX.XX/XXX.XX

by taking advantage of compressed representations. Using these structures, we obtain the following contributions.

(1) Our structure enables retrieving all time points of any given BGP (via LTJ), allowing any possible variable binding order, with (at least) the same complexity guarantees that time-first or join-first algorithms offer. This already improves the current state of the art with respect to worst-case optimal joins over temporal graphs.

(2) We show that we can answer much more general queries, which we call *temporal* BGPs (tBGPs), under the same guarantees. These queries allow for retrieving different time intervals for different parts of the query, and to apply filters restricting these intervals.

(3) We can answer point-in-time or interval queries that return all answers valid in a given timestamp or time period with stronger guarantees: we do it in wco time with respect to the graph restricted to that point or interval rather than the complete graph. This provides a further speedup for point-in-time or interval queries, which are key primitives of the versioning functionalities of the SQL 2011 standard and several graph database proposals [14, 19, 28, 31].

(4) We provide a prototype implementation and show that our improvements translate to practice, solving realistic temporal queries within milliseconds while using reasonable index space.

## 2 RELATED WORK

Temporal query languages have been extensively studied in relational and graph databases, leading to temporal extensions of SQL:2011 [25] and to proposals for RDF and graph query languages such as T-SPARQL [19], stSPARQL [24], and SPARQL-ST [32]. Systems such as GLENDa [31] and RDF archives [14, 18] further support querying historical graph snapshots and tracking data evolution over time. Our work is complementary to these efforts, focusing not on language design but on worst-case-optimal evaluation algorithms for temporal graph patterns.

Despite the advances in temporal languages and data models, the body of work in algorithms supporting these query extensions is much thinner. Berberich et al. focus on temporal extensions to document databases (see e.g. [3, 8]), but these algorithms cannot be directly applied into graph patterns. Work on temporal constructs or temporal algebras, for relations or graphs, can lead directly to algorithms [9, 15, 28], but these are based on traditional techniques and not wco joins. In addition, all of these contributions emphasize the need for tailored evaluation strategies considering both the temporal constructs and the structural complexity of the data. Relatedly, Abo Khamis et al. [2] study the complexity of Boolean conjunctive queries with *intersection joins*, which conceptually align with temporal constraints that combine matches by intersecting time domains. Their techniques can be adapted to temporal joins, typically incurring an additional logarithmic indexing factor, while our work targets general temporal graph patterns in linear space.

Recent work on algorithms instead focuses on how to evaluate graph patterns under specific data-time constraints, such as retrieving BGPs that occur in the same time point within an interval [39], answers that exist for a long period of time [26, 30], for the longest period [33], or answers where edges respect time constraints, in the sense that every match for a triple  $(s, p, o)$  must satisfy that  $s$  exists before  $o$  [17, 22]. Unfortunately, none of these algorithms can be applied directly in our case because they are designed for specific time

constraints. In contrast, our work focuses on answering arbitrary temporal graph patterns, which is a more general problem: all of these specific time constraints can be expressed as temporal graph patterns, and the expressive power of temporal graph patterns goes well beyond these constraints.

Our work is closest to the approach of Hu et al. [21], which analyzes both join-first and time-first strategies for wco joins over temporal graphs. However, as we mentioned, our approach can handle any variable binding ordering, more general queries, and interval or point queries with better guarantees. One thing to note is that their algorithm always involves a sweep over the time domain, which takes  $\Theta(N)$  time. This corresponds, in our work, to what we obtain by binding the time variable first. Even though our techniques lead to implementing LTJ with all variable binding plans available (including time-first), one possible advantage of the work by Hu et al. is that, as they first sweep over time and then process the query in two independent steps, this gives them the chance to use more sophisticated algorithms for the (non-temporal) query part, such as EmptyHeaded [1], which uses the Generalized Hyper-tree Decomposition (GHD) of queries. Moreover, as they show in their paper, their idea of only obtaining the solutions where the disappearing edge participates yields better results on hierarchical queries. Finally, their paper also proposes a cross between time-first and join-first combined with GHD-based algorithms, which again may work better or worse, depending on the instance, than our query plans. We discuss in the Conclusions how our algorithms can be adapted to work under GHD-like time guarantees as well.

## 3 MODEL

Here we describe temporal graphs, queries, and their AGM bound.

### 3.1 Temporal graphs and queries

As is usual in the literature, we regard a temporal graph  $G$  as a set of labeled edges  $s \xrightarrow{p} o$ , or triples  $(s, p, o)$ , that exist during a given time interval  $[ti, tf)$  (see, e.g., [19, 32]).

**DEFINITION 1.** Let  $\mathcal{U}$  be an infinite set of values and  $(\mathcal{T}, \leq)$  a totally ordered time domain, disjoint from  $\mathcal{U}$ . A temporal graph  $G$  is a set of  $N := |G|$  tuples  $(s, p, o, [ti, tf))$ , where  $s, p, o$  are elements from  $\mathcal{U}$ , and  $ti, tf$  are elements from  $\mathcal{T}$ , with  $ti < tf$ , so that  $[ti, tf)$  represents an interval in the time domain. We also define  $\mathcal{U}_G := \{s, p, o \mid \exists ti, tf, (s, p, o, [ti, tf)) \in G\}$  and  $\mathcal{T}_G := \{[ti, tf) \mid \exists s, p, o, (s, p, o, [ti, tf)) \in G\}$  as the set of elements and time instants mentioned in tuples of  $G$ .

Intuitively, a tuple  $(s, p, o, [ti, tf))$  in  $G$  represents that the triple  $(s, p, o)$  is valid in any time  $t$  such that  $ti \leq t < tf$ . For simplicity we assume that temporal graphs do not have redundant tuples: if  $G$  has tuples  $(s, p, o, [ti, tf))$  and  $(s, p, o, [ti', tf'))$ , then it must hold  $tf < ti'$  or  $tf' < ti$ . This condition ensures that there is a unique way to represent temporal graphs. Furthermore, since  $\mathcal{U} \cap \mathcal{T} = \emptyset$ , node identifiers and labels cannot be time instants.

Figure 1 (left) shows an example temporal graph  $G$  with  $\mathcal{U}_G = \{1, \dots, 10\}$  and  $\mathcal{T}_G = \{1, \dots, 4\}$ . It describes the evolution of a social network where nodes 1 to 7 are persons and node 8 is a group; The graph contains edges follows (solid arrows) and edges subscriber (dashed arrows). Arrows show their interval of validity

$[ti, tf]$ . Ignore for now the second graph,  $\hat{G}$ . To its right, we show the snapshots  $G_1$  and  $G_3$  of the valid edges at instants 1 and 3.

Our focus is on graph patterns, which are the foundation of most query languages used in industry, including SPARQL, CYPHER and GQL [4, 16]. To support temporal operations in graph patterns, we add temporal variables to patterns, and temporal filters.

**DEFINITION 2.** Let  $\mathcal{V}_u, \mathcal{V}_t$  be two disjoint sets of variable symbols, additionally disjoint from  $\mathcal{U}$  and  $\mathcal{T}$ . A temporal basic graph pattern (tBGP) is a set of tuple patterns  $(x, y, z, w)$ , where each  $x, y$ , and  $z$  are in  $\mathcal{U}_G \cup \mathcal{V}_u$ , and  $w$  is in  $\mathcal{T}_G \cup \mathcal{V}_t$ , plus optional comparison clauses of the form  $w_1 \leq w_2$ , where  $w_1, w_2 \in \mathcal{T}_G \cup \mathcal{V}_t$ .

We note, in particular, that tBGPs can only mention time instants that exist in  $\mathcal{T}_G$ . This is not really a restriction since the edges of  $G$  at a time  $t \in \mathcal{T}$  are exactly the same as those at time  $t'$ , where  $t'$  is the predecessor of  $t$  in  $\mathcal{T}_G$ . Queries can then be translated into valid tBGPs: clauses  $w_1 \leq w_2$  stay valid after mapping values of  $\mathcal{T}$  to their predecessors in  $\mathcal{T}_G$ . We also assume that those clauses are satisfiable, and do not form cycles (like  $w_1 \leq w_2 \leq w_1$ , as those are expressed by collapsing the variables in the cycle into one).

The semantics of tBGPs is given by assignments, called *solutions*.

**DEFINITION 3.** A solution to a tBGP with variables  $\mathcal{V}_u$  and  $\mathcal{V}_t$  over a temporal graph  $G$  is an assignment  $f : \mathcal{U}_G \cup \mathcal{V}_u \rightarrow \mathcal{U}_G$  and  $f : \mathcal{T}_G \cup \mathcal{V}_t \rightarrow \mathcal{T}_G$ , such that  $f(u) = u$  if  $u \in \mathcal{U}_G$  and  $f(t) = t$  if  $t \in \mathcal{T}_G$ . Furthermore, for each tuple pattern  $(x, y, z, w)$  in the tBGP, there must exist a tuple  $(f(x), f(y), f(z), [ti, tf]) \in G$  where  $ti \leq f(w) < tf$ . Finally, for each clause  $w_1 \leq w_2$  it must hold  $f(w_1) \leq f(w_2)$ .

Note that the time component of our solutions must also belong to  $\mathcal{T}_G$ . This is a form of compacting the (possibly infinite) set of all the solutions in  $\mathcal{T}$ : a solution with time  $t \in \mathcal{T}_G$  stays valid for all the times  $t'' \in \mathcal{T}$  such that  $t \leq t'' < t'$ , where  $t'$  follows  $t$  in  $\mathcal{T}_G$ . We investigate the problem of solving a tBGP  $Q$ , defined as follows.

**DEFINITION 4.** Given a temporal graph  $G$ , the problem of solving a tBGP  $Q$  is that of computing the set  $Q(G)$  of all the solutions to  $Q$ .

Intuitively, each solution  $f \in Q(G)$  represents an assignment that maps each tuple pattern  $(x, y, z, w)$  in the tBGP to a triple  $(f(x), f(y), f(z))$  in  $G$  such that this triple is valid in time  $f(w)$ . For example, for  $t_1, t_2$  in  $\mathcal{T}_G$ , query  $(x, y, z, w), t_1 \leq w, w \leq t_2$  returns all triples  $(s, p, o)$  that are valid in any time point between  $t_1$  and  $t_2$ . In this case, the answers are given as a set  $(s, p, o, t'_1), (s, p, o, t'_2), \dots, (s, p, o, t'_\ell)$ , where  $t'_1, \dots, t'_\ell \in \mathcal{T}_G$  are all time points between  $t_1$  and  $t_2$  in which  $(s, p, o)$  exists. One can always pack the answers into time intervals if required.

In Figure 1, a tBGP  $Q = \{(x, \text{follows}, z, w), (y, \text{follows}, z, w), (8, \text{subscriber}, z, w)\}$  retrieves pairs  $(x, y)$  of followers of the same person  $z$  that is subscribed to group 8, all happening at the same time  $w$ . Two answers from  $Q(G)$  are  $\langle x = 3, y = 6, z = 4, w = 2 \rangle$  and  $\langle x = 3, y = 6, z = 4, w = 3 \rangle$ . In packed form, they can be written as  $\langle x = 3, y = 6, z = 4, w \in [2, 4) \rangle$ .

The selection of tBGPs as our language is justified as the key primitive of temporal relational calculus [13], the algebraic building block of most temporal database languages (see e.g. [19, 32, 34]). Hence, tBGPs are a fundamental querying functionality of temporal database languages, much like standard basic graph patterns are for (non-temporal) graph query languages. We compare tBGPs to other query languages in Section 3.4.

### 3.2 Temporal worst-case optimal algorithms

The AGM bound requires us to reason about cardinalities of relations. In order to do this, we use the notion of the point graph  $\hat{G}$  of  $G$ : the AGM bound of a query  $Q$  over a graph is defined as the maximum size of the answer of the evaluation of  $Q$  over any graph whose point-wise representation has the same number of tuples.

**DEFINITION 5.** The point-based representation of  $G$  is defined as

$$\hat{G} = \{(s, p, o, t) \mid \exists ti, tf, (s, p, o, [ti, tf]) \in G, t \in [ti, tf) \cap \mathcal{T}_G\}.$$

The times  $t$  present in  $\hat{G}$  are only those constants  $ti, tf \in \mathcal{T}$  that appear as interval extremes in  $G$  (potentially on another triple); see the second graph in Figure 1. As explained, it is unnecessary to consider other times in the domain where  $G$  does not change. As a result, we have the bound  $|\hat{G}| \leq |G| \cdot |\mathcal{T}_G| \in O(N^2)$ , which is tight when the edges of  $G$  are valid for long time intervals and many different time instants exist in  $G$ .

**DEFINITION 6.** The AGM bound  $Q^*$  of a tBGP  $Q$  for a temporal graph  $G$  is  $Q^* := \max\{|Q(G')|, |\hat{G}'| \leq |\hat{G}|\}$ , over temporal graphs  $G'$ . An algorithm computing  $Q(G)$  is worst-case optimal if it takes time  $O(Q^*)$ , possibly with data-agnostic and/or polylog factors of  $|G|$ .

The following theorem states our main result.

**THEOREM 1.** Let  $G$  be a temporal graph with  $N$  tuples. Then, there exists a data structure using  $O(N)$  space that can compute  $Q(G)$  for every tBGP  $Q$  with  $m$  tuples in time  $O(Q^* m \log N)$ .

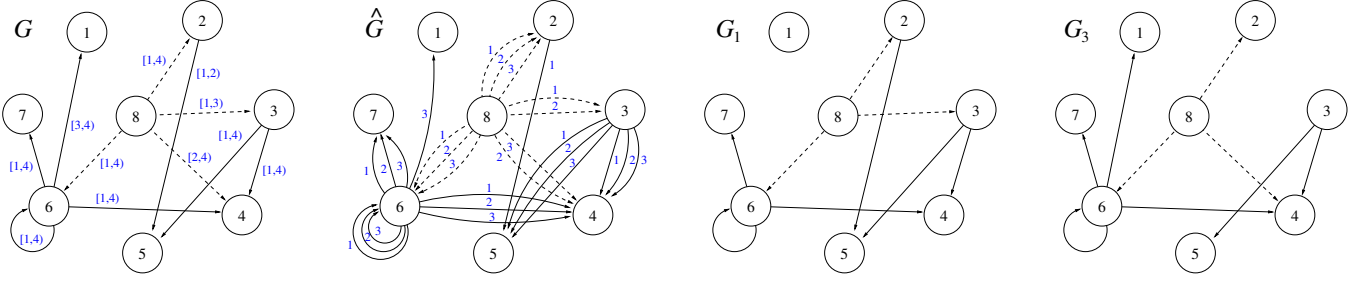
The wco algorithm in Theorem 1 can be easily obtained by running a wco algorithm directly on  $\hat{G}$ , interpreting the tuples of the tBGP  $Q$  as quad-patterns (let us disregard the time clauses for simplicity in this discussion). The disadvantage is, of course, that  $\hat{G}$  is a bloated representation of  $G$ , which can require quadratic space. We could, alternatively, build  $\hat{G}$  from  $G$  and index it on the fly at query time, but this translates the  $O(N^2)$  factor to the query time and working space. This is unacceptable in most practical cases.

Our representation of Theorem 1 actually *simulates* the use of the LTJ algorithm [35] on  $\hat{G}$ , but uses  $G$  in native form within  $O(N)$  space, without converting it to  $\hat{G}$ , and therefore without any of the space or time penalizations we discussed. Our algorithm is not only wco, but can also simulate LTJ with any desired variable elimination order (VEO), exactly as if run on  $\hat{G}$ . The ability to choose specific VEOs is crucial to obtain good practical performance, even if in principle all the VEOs yield wco times [35].

### 3.3 Point & interval queries: better guarantees

We obtain even stronger bounds for point-in-time and point-in-interval queries, which ask for all answers of a BGP valid at a given time point  $t$ , or at some point during the interval  $[ti, tf]$ . We can also query for BGPs that were valid along a whole time interval. To state these results, we define some (non-temporal) labeled graphs, derived from slicing a temporal graph across the time domain. We use  $|G|$  to denote the number of triples in a non-temporal graph.

**DEFINITION 7.** For any  $t \in \mathcal{T}_G$ ,  $G_t$  is the set of triples  $(s, p, o)$  such that there exists a tuple  $(s, p, o, [ti, tf])$  in  $G$  with  $ti \leq t < tf$ . For any  $t_1 < t_2$  in  $\mathcal{T}_G$ ,  $G_{[t_1, t_2]}$  is the set of triples  $(s, p, o)$  such that there exists a tuple  $(s, p, o)$  in  $G_t$  for every  $t \in \mathcal{T}_G$  with  $t_1 \leq t < t_2$ , and  $G_{[t_1, t_2]}$



**Figure 1: A temporal graph  $G$ , its point-based representation  $\hat{G}$ , and two snapshots at times  $t = 1$  and  $t = 3$ . Solid and dashed arrows are labeled with two different predicates: follows and subscriber, respectively.**

is the set of triples  $(s, p, o)$  such that there exists a tuple  $(s, p, o)$  in  $G_t$  for some  $t \in \mathcal{T}_G$  with  $t_1 \leq t < t_2$ .

Let us first focus on point-in-time and point-in-interval queries. In our notation, a point-in-time query amounts to solving classic BGP pattern matching on graph  $G_t$ , and a point-in-interval query requires matching BGPs on  $G_{[t_1, t_2]}$ , where  $t$  and  $t_1, t_2$  are given at query time. Using our data structure we can solve these problems in wco time with respect to the sizes of  $G_t$  or  $G_{[t_1, t_2]}$ , which is the best one can hope for: the algorithm is wco on the labeled graph that has exactly the triples we want to consider. This also improves the time shown in Theorem 1 since all such graphs are smaller than  $\hat{G}$ . Concretely, we show the following in Section 7.

**THEOREM 2.** *Let  $G$  be a temporal graph with  $N$  tuples. Then, there is a data structure using  $O(N)$  space that allows the following:*

- Answer BGPs  $Q$  with  $m$  tuples on the labeled graph  $G_t$  for any time instant  $t$  given with  $Q$ , in  $O(Q_t^* m \log N)$  time, where  $Q_t^*$  is the maximum number of solutions for  $Q$  in some labeled graph with at most  $|G_t|$  triples, and
- Answer BGPs  $Q$  with  $m$  tuples on the labeled graph  $G_{[t_1, t_2]}$  for any time interval  $[t_1, t_2]$  given with  $Q$ , in  $O(Q_{[t_1, t_2]}^* m \log N)$  time, where  $Q_{[t_1, t_2]}^*$  is the maximum number of solutions for  $Q$  in some labeled graph with at most  $|G_{[t_1, t_2]}|$  triples.

In Section 7 we also show that our data structure leads to instance-optimal algorithms when matching a single triple pattern, and can also be used to *list* all time intervals where these matches existed.

Next are duration queries, with two flavors. First, we can specifically input a BGP and time points  $t_1, t_2$  at query time, and require all tuples that existed during the whole interval, or, in other words, to query graph  $G_{[t_1, t_2]}$ . We can also input a duration  $\delta$  instead of the interval, in which case we want to look for answers that are valid for at least  $\delta$  time. We show the following in Section 7.2.

**THEOREM 3.** *Let  $G$  be a temporal graph with  $N$  tuples. Then, there is a data structure using  $O(N)$  space that allows the following:*

- Solve BGPs  $Q$  with  $m$  triples on the labeled graph  $G_{[t_1, t_2]}$  for any time interval  $[t_1, t_2]$  given with  $Q$ , in  $O(Q_{[t_1, t_2]}^* m \log N)$  time, where  $Q_{[t_1, t_2]}^* = \min_{t_1 \leq t < t_2} Q_t^*$ .
- Solve BGPs  $Q$  with  $m$  triples, reporting all the time intervals of length at least  $\delta$  where  $Q$  holds in  $G$  for any duration  $\delta$  given with  $Q$ , in time  $O(Q_\delta^* m \log N)$ , where  $Q_\delta^*$  is the maximum

number of solutions for this query in the subgraph of  $G$  formed by all tuples with duration  $\delta$  or more.

### 3.4 On the choice of temporal BGPs

Temporal BGPs play a role for temporal graphs analogous to conjunctive queries (CQs) for relational data: they form the pattern-matching core underlying richer temporal query languages [13].

The study of tBGPs is further justified by the fact that algorithms for standard conjunctive query evaluation cannot be directly translated to the temporal setting, because the language of tBGPs can be more expressive than CQs over a standard relational encoding of temporal graphs (see Appendix A in the extended version). On the other hand, if  $G$  is expanded into its point-based representation  $\hat{G}$ , then every tBGP corresponds to a CQ with inequalities evaluated over  $\hat{G}$  (see the extended version). This further justifies tBGPs as the CQ analogue in our setting. Since the size of  $\hat{G}$ , for a graph  $G$  with  $N$  tuples, can be of order  $N \cdot |\mathcal{T}_G|$ , materializing  $\hat{G}$  to process tBGPs as relational CQs is infeasible. Notice that our index structures can correctly evaluate tBGPs over  $\hat{G}$  using only  $O(N)$  space.

## 4 OVERVIEW OF OUR SOLUTION

Our solution is to build a data structure for temporal graphs in which we can run the Leapfrog TrieJoin algorithm with any variable ordering to compute joins in wco time, and where the total space is linear with respect to the number of tuples of the temporal graph.

For a temporal graph  $G$ , we replace the  $ti$  and  $tf$  components in  $G$  by their rank in  $\mathcal{T}_G$ , turning them into integers in  $[0, T)$  where  $T = |\mathcal{T}_G|$  (we also maintain a dictionary to enable switching between actual time constants and integers in  $[0, T)$ ). We further map the set of node identifiers and labels to the integer interval  $[0, U)$ , where  $U = |\mathcal{U}_G|$ . We handle those numeric identifiers using binary tries.

In what follows we define the Leapfrog TrieJoin algorithm and (versioned) binary tries, the two basic building blocks of our solution. Then, in Section 5, we explain how to use these building blocks to obtain a data structure that achieves the desired wco optimality, but with  $O(N \log N)$  space. In Section 6 we explain how to further reduce this to a structure using linear space, and we then explain how to use this structure for querying.

### 4.1 Leapfrog Triejoin

We describe the Leapfrog Triejoin (LTJ) algorithm [35], in the version that is adapted for solving BGPs on labeled graphs [20]. LTJ

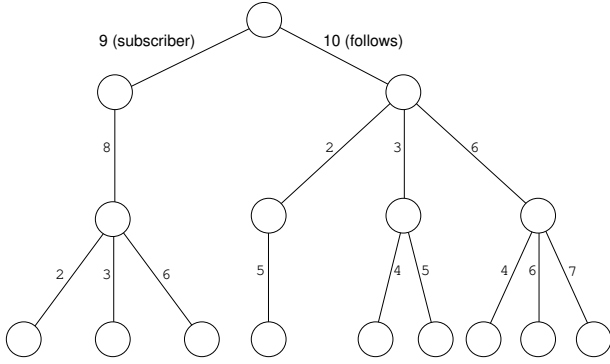


Figure 2: The LTJ trie of order pso for the graph  $G_1$  of Figure 1.

requires that the triples  $(s, p, o)$  are represented as tries, which we will call *LTJ tries*, in the 6 possible orders of the components  $s$ ,  $p$ , and  $o$ . Tries are labeled trees where no two children of a node have the same label, and represent all the strings that can be read by concatenating the labels of their root-to-leaf paths.

Each LTJ trie stores the components  $s, p, o$  in some order, as strings of length 3. We call these orders  $s, p, o$ ,  $s, o, p$ ,  $p, s, o$ ,  $p, o, s$ , and  $o, s, p$ . For example, the trie for the order  $pso$  has one root-to-leaf path with consecutive labels  $p, s, o$  for each triple  $(s, p, o)$  in the graph (see Figure 2). Each LTJ trie has height 3 and exactly  $N$  leaves.

Let  $Q = \{t_1, \dots, t_m\}$  be a BGP (i.e., a set of triple patterns) and  $\{x_1, \dots, x_o\}$  its set of variables. LTJ carries out  $v$  iterations, “eliminating” one variable at a time. The order LTJ chooses to eliminate the variables is known as the *variable elimination order* (VEO).

Each triple pattern  $t_i$  is associated with one of the 6 tries, say  $\tau_i$ . We will say that a level  $\ell$  of  $\tau_i$  corresponds to a constant  $c$  (or to a variable  $x$ ) if  $t_i$  contains  $c$  (or  $x$ ) at the position corresponding to level  $\ell$  in  $\tau_i$ . To be a valid trie for triple  $t_i$ , the first levels of  $\tau_i$  (i.e., closest to the root) must correspond to the constants of  $t_i$ , and the subsequent levels must correspond to the variables of  $t_i$  in the order consistent with the VEO. For example, on the graph  $G_1$  of Figure 1, if  $t_i = (x, \text{follows}, z)$  and the VEO eliminates  $x$  and then  $z$ ,  $\tau_i$  must be the trie of the order  $pso$  shown in Figure 2 (this is why LTJ needs the tries in various orders). Its first level corresponds to  $p = \text{follows}$ , the second to  $s = x$ , and the third to  $o = z$ .

The first step of LTJ is to descend, from the root of each trie  $\tau_i$ , by the constants of  $t_i$ . It then starts the variable elimination step. Say that the chosen VEO is  $x_1, \dots, x_o$ . LTJ then finds each value  $c$  that appears as a child of the current node in every trie  $\tau_i$  whose next level corresponds to variable  $x_1$ . For each such  $c$ , LTJ *binds*  $x_1 := c$ , descends by  $c$  in all the corresponding tries  $\tau_i$ , and goes on recursively with  $x_2$ . Once we have bound all variables in this way, the set of bindings for  $x_1, \dots, x_o$  is a new solution for  $Q$ .

The values  $c$  are found by intersecting the children of the current nodes  $v_i$  in all the suitable tries  $\tau_i$ ; see Algorithm 1. LTJ performs the intersection using the primitive  $\text{leap}(v_i, x)$ , which finds the smallest constant  $x' \geq x$  within the children of the current node  $v_i$  in trie  $\tau_i$ . If  $\text{leap}$  is executed in  $O(\log N)$  time (e.g., using binary search) then LTJ obtains the wco time complexity  $O(Q^* m \log N)$ .

```

 $x \leftarrow 0; l \leftarrow 0;$ 
while true do
   $x \leftarrow \text{leap}(v_l, x);$ 
   $i \leftarrow (l + 1) \bmod k;$ 
  while  $i \neq l$  do
     $x' \leftarrow \text{leap}(v_i, x);$ 
    if  $x' > x$  then  $x \leftarrow x'; l \leftarrow i;$ 
     $i \leftarrow (i + 1) \bmod k;$ 
  end
  if  $x = +\infty$  then break;
  report  $x$ ;
   $x \leftarrow x + 1;$ 
end

```

Algorithm 1: The LTJ iterator reports all common children of LTJ trie nodes  $v_0, \dots, v_{k-1}$ .  $\text{leap}(v, x)$  returns the next value  $\geq x$  descending from LTJ trie node  $v$ , or  $+\infty$  if none exists.

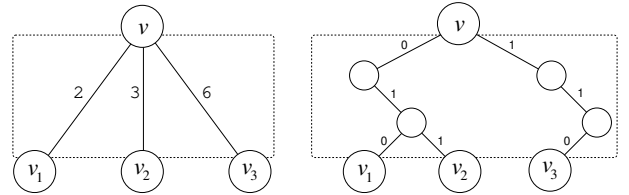


Figure 3: On the left, an LTJ trie node  $v$  with three children: values 2, 3, and 6. On the right, the BT for that set of children, using  $\ell = 3$  bits. The BT represents the binary strings  $2 = 010_2$ ,  $3 = 011_2$ , and  $6 = 110_2$ . The root of the BT is  $v$  and its leaves are the corresponding children of  $v$  in the LTJ trie.

## 4.2 Binary Tries

Assume that we aim to represent each  $G_t$  as a labeled graph, for the consecutive values  $t \in [0, T)$ . We now describe a technique inspired by versioned data structures, beginning with binary tries.

A *binary trie* (BT) is a binary tree that represents a set of binary strings, each string being a root-to-leaf path in the trie. If the sequence of left-right turns from root to the leaf are interpreted as 0 and 1, respectively, we obtain the string represented by the leaf. We assume no binary string is a prefix of another.

We will use BTs to represent the sets of children of any node  $v$  of the LTJ tries. Those children are numbers in  $[0, U)$ , which we will interpret as binary strings of length  $\ell = \lceil \log_2 U \rceil$ , reading from the most to the least significant digit. Figure 3 illustrates the BT for the LTJ trie node that descends from  $\langle \text{subscriber}, 8 \rangle$  in Figure 2.

BTs can simulate the search for the first child of  $v$  with value  $\geq x$  in  $O(\ell) = O(\log N)$  time, as follows. We start at the BT root, with height  $h := \ell$ . If the  $h$ -th highest bit of  $x$  (seen as an  $\ell$ -bit number) is a 1, then we continue recursively by the right child of the BT, decrementing  $h$ . Otherwise, we do as follows. First, we try to find the answer on the left branch, also decrementing  $h$ . If that search returns an answer, we return it too. If not, we return the leftmost leaf of the right child of the node. When we arrive at a leaf, its string represents the first value  $\geq x$  in the trie. This takes  $O(\ell)$  time because it can descend by both children of a node only once

in the whole process. For a pseudocode see the extended version (Algorithm 2 in Appendix B).

BTs then support the LTJ leap operation within the same  $O(\log N)$  time factor penalty of binary searches. As a consequence, we can use a BT as a local structure to represent the set of children of every LTJ trie node (instead of just a plain array of increasing identifiers), and retain the same wco time  $O(Q^* \log N)$  on labeled graphs.

### 4.3 Versioned Binary Tries

In general terms, we will aim to represent all the graphs  $G_t$  with the same tries. When representing  $G_t$ , then, we will have LTJ trie nodes  $v$  that will be very similar to the node  $v$  for  $G_{t-1}$ . Concretely, the node  $v$  in  $G_t$  may have a few inserted or deleted children with respect to the node  $v$  in  $G_{t-1}$ . To efficiently represent the children of those LTJ trie nodes  $v$ , we will use *versioned binary tries* (VBTs). A VBT represents a BT as a set of updates with respect to a *reference* BT. Instead of explicitly storing all the nodes of the BT it represents, the VBT records only the root-to-leaf paths that change with respect to the reference BT. Changes refer to newly inserted or deleted binary strings. The BT subtrees that do not change, instead of being duplicated, are pointed to from the corresponding nodes in the newly created paths to the reference BT. Figure 4 illustrates the successive versions of the subtree given in Figure 3.

Note that a top-down traversal on a VBT is identical to that on a standard BT, so we can run the LTJ intersections on the VBTs as well, in  $O(\log N)$  time per leap. Note how, in Figure 4, we can traverse the VBT of the third version exactly as if it were a BT.

We use VBTs to represent a sequence of versions of a node’s children. The first version in time is represented as a standard BT. Each new version is a VBT encoded with respect to the preceding BT. From the third VBT onwards, the preceding BT is also represented as a VBT, so pointing to a subtree of the preceding BT may actually correspond to pointing to an earlier BT pointed to by the preceding VBT. This is shown on the third version in Figure 4.

Since the paths are of length  $O(\log N)$ , it follows that VBTs require  $O(\log N)$  space per update they record with respect to a previous BT. VBTs will be part of our solution, as described next.

## 5 OUR REPRESENTATION

In abstract terms, we regard each tuple  $(s, p, o, [ti, tf])$  of  $G$  as if it were  $tf - ti$  standard quads  $(s, p, o, t)$ , one per integer  $t \in [ti, tf]$ ; recall that we have mapped  $\mathcal{T}$  to  $[0, T)$ . This is  $\hat{G}$ , which as explained may have  $\Theta(N^2)$  quads, but we will manage to represent them all within  $O(N \log N)$  space (later, we will reduce the space to  $O(N)$ ).

With the quads model, the tBGPs can be solved directly by using the LTJ algorithm on the LTJ tries storing the  $4!$  permutations of  $\{s, p, o, t\}$ . The algorithm is then wco with respect to this model. In particular, consider a tBGP where the time component is a single constant  $t$  for all the triple patterns. Then, using an LTJ trie with an order that starts with  $t$ , we descend by the child  $t$  of the root and can solve the tBGP in time  $O(Q_t^* \log N)$ , where  $Q_t^*$  is the AGM bound for the corresponding BGP on the graph  $G_t$ . As another example, consider the same tBGP where now  $t$  is a variable. By solving it on an LTJ trie that starts with the  $t$  component, we can mimic the so-called “time-first” approach in previous work [21]; by using a trie that ends with the  $t$  component, we mimic the so-called

“join-first” strategy. We can use more general strategies, however, by putting the time component elsewhere in the order.

In this scenario, the clauses  $w_1 \leq w_2$  are handled as follows. If  $w_1$  is bound before  $w_2$ , then it will have assigned a value  $f(w_1) = t_1$  each time  $w_2$  is bound. At this moment we restrict the LTJ intersections for  $w_2$  so that the values stay within  $[t_1, T)$ . If, instead,  $w_2$  is bound to  $f(w_2) = t_2$  before  $w_1$ , we enforce that the values of  $w_1$  be within  $[0, t_2]$ . This is still wco, as it is equivalent to materializing a table with all the pairs  $(w_1, w_2)$  with  $w_1 \leq w_2$  and including it in the LTJ algorithm. Since all the time instants appear in both columns, LTJ will not restrict the first bound variable; the second one will be restricted exactly as described.

To do all this within  $O(N \log N)$  space, we introduce a special representation for the LTJ tries. Their abstract form will still be a trie of height 4, with one level per tuple component. The difference is in the way we represent the children of the nodes. Given a particular permutation, say  $(s, t, p, o)$ , we distinguish the levels before the time component (in our example, the first level,  $s$ ), the time level, and the levels after the time component (levels 3 and 4,  $p$  and  $o$ , in our example). Note that which levels are before and after the time level depends on the permutation we are storing.

The levels before the time level can be implemented in traditional form, e.g., as an array of increasing child values. We now describe our implementation of the time level and its subsequent levels.

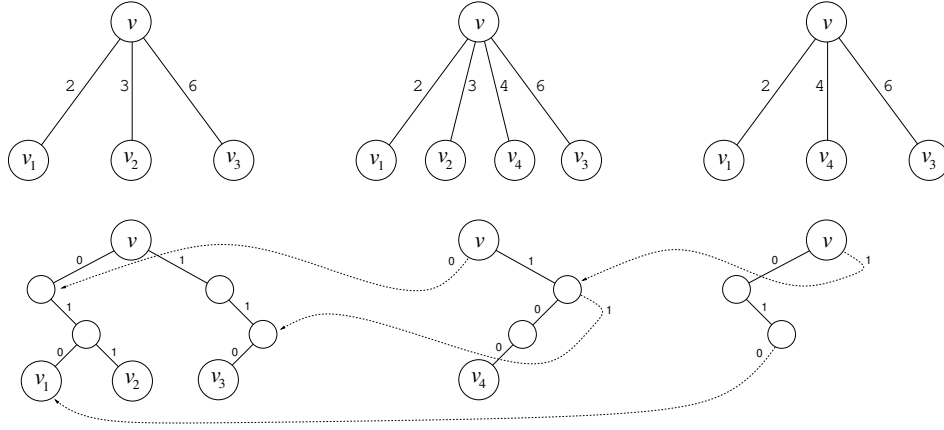
### 5.1 The time level

The time level will be represented in a form closer to the tuples of  $G$  (with the times already converted to integers in  $[0, T)$ ) than to the set of quads of  $\hat{G}$ . This level will be an ordered sequence of disjoint time intervals, stored in classical form (e.g., an array). Let  $v$  be an LTJ trie node whose children belong to the time level, and  $[t_{i1}, t_{f1}), [t_{i2}, t_{f2}), \dots$  be the time intervals of all the tuples stored in the LTJ subtree of  $v$  (this is a subset of all the time intervals in  $G$  and they can overlap; for example only the time instants  $t = 1$  and  $t = 4$  exist below the node of  $\langle \text{follows}, 3 \rangle$  in the pso trie of our graph of Figure 1). Then consider the list  $\langle t_1, t_2, \dots \rangle$  containing all the values  $t_{ik}$  and  $t_{fk}$ , sorted in increasing order and with equal values removed. The intervals forming the children of  $v$  are then in principle  $[t_1, t_2), [t_2, t_3), \dots$ , which covers the whole universe  $[0, T)$ . We remove, however, intervals with no descending tuples. The surviving intervals are then of the form  $[ts_1, te_1), [ts_2, te_2), \dots$ , where each  $[ts_i, te_i)$  is equal to some  $[t_k, t_{k+1})$ .

Note that every tuple in  $G$  produces at most 2 entries below some node  $v$  in the time level of each LTJ trie, so the total number of nodes in this level is at most  $2N$  per LTJ trie. For example, in the LTJ trie for order  $(s, t, p, o)$ , a tuple  $(s, p, o, [ti, tf])$  will generate, below the node  $v$  that represents  $s$ , a time  $ti$  where the event of inserting  $p : o$  will occur, and a time  $tf$  where the event of deleting  $p : o$  will occur, as we detail next.

### 5.2 Levels after the time component

We will use BTs to represent the children of all LTJ trie nodes below the time level. Consider an LTJ trie node  $v$  whose children, at the time level, are  $[ts_1, te_1), [ts_2, te_2), \dots$ . We will *concatenate* all the binary strings corresponding to all the levels that follow the time level. For example, if the LTJ trie follows the order  $(s, t, p, o)$ , then



**Figure 4: At the top, three consecutive versions of an LTJ trie node  $v$ . At the bottom, their representation using VBTs. Note that the value 4 appears in the second version, whereas 3 disappears in the third version. The pointers to preceding versions are shown with dashed lines; note in particular the pointer from the third to the first version.**

the descendants of the node  $[ts_i, te_i]$  will be arranged in a single BT holding binary strings of length  $2\ell$ , formed by concatenating the  $p$  and the  $o$  values (i.e.,  $p : o := 2^\ell p + o$ ) of all the tuples that must be stored below  $v$  and exist at time  $ts_i$ . Per our construction, this set of tuples does not vary within the time interval  $[ts_i, te_i]$ .

Figure 5 gives an example for the node descending from  $\langle \text{follows} \rangle$  in the trie ordered pso of our graph of Figure 1, at time instants  $t = 1$  (shown in Figure 2) and  $t = 3$ . Note that concatenating the components is almost immaterial with respect to using different BTs for the children of every LTJ trie node. In our example, the BT node representing each prefix  $p$  (at depth  $\ell$ ) becomes the root of the subtree representing each component  $o$  of the concatenations  $p : o$ . We can then interpret the node of  $p$  as the LTJ trie node obtained by descending by  $p$  from  $v$ , and the subtree with the  $o$  components as the BT of its children. Our arrangement, however, is more convenient for the versioning we require next.

Let us consider how we store the elements formed by concatenating the remaining attributes of the tuples below  $v$ . We first build the basic BT for the node  $[ts_1, te_1]$ , containing all the elements whose tuples exist in time  $ts_1$ . This set of tuples does not vary until time  $te_1$ . This BT is the child of the node  $[ts_1, te_1]$  of the time level.

Now consider the next interval,  $[ts_2, te_2]$ . If  $ts_2 > te_1 + 1$ , then the intervals spanning  $[te_1, ts_2]$  were empty, and we simply build a new BT for  $[ts_2, te_2]$  containing only the new elements that appear at time  $ts_2$ . Otherwise, there exists a nonempty BT that precedes the BT for  $[ts_2, te_2]$ , so we represent this BT as a VBT, whose reference is the BT for  $[ts_1, te_1]$ . This VBT must represent two events:

- (1) Consider a tuple  $(s, p, o, [ti, tf])$  of  $G$  where  $(s, p, o)$  must be inserted below the LTJ trie node  $v$  and such that  $ti = ts_2$ . This tuple must be added to the VBT.
- (2) Similarly, each tuple  $(s, p, o, [ti, tf])$  belonging to the subtree of  $v$  and such that  $tf = ts_2$  must be removed from the VBT.

We create new paths in the VBT of  $[ts_2, te_2]$  as described in Section 4.2, to account for those events. Each event requires creating  $O(\log N)$  nodes in the BTs or the VBTs. Note again that each graph tuple  $(s, p, o, [ti, tf])$  generates two events: the insertion of  $(s, p, o)$

at  $ti$  and its deletion at  $tf$ . Therefore, the trie represents at most  $2N$  events, each of which induces  $O(\log N)$  new nodes in the VBTs. The total space is thus  $O(N \log N)$ .

### 5.3 The intersections

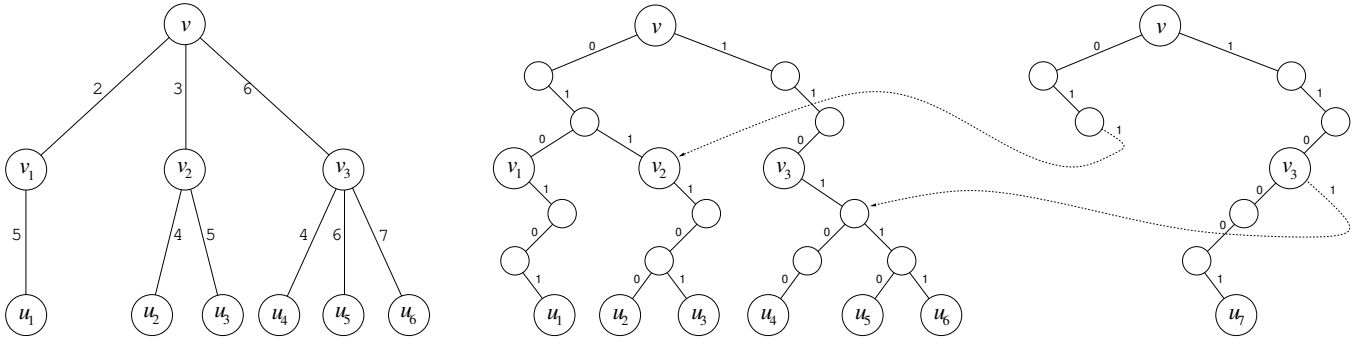
Intersections work exactly as in LTJ over BTs, except at the time level. At this level, each node represents an interval, which must be handled during intersection: each interval  $[ts_i, te_i]$  stands for all the time instants  $ts_i \leq t < te_i$ . We modify the list intersection algorithm of LTJ so as to assume that all those time instants  $t$  are explicitly represented and that copies of the BT of  $[ts_i, te_i]$  descend from all those implicit time instants  $t$ : To find the first time  $t^* \geq t_0$ , we look for  $t_0$  and, (1) if we find some  $ts_i \leq t_0 < te_i$ , we answer  $t^* := t_0$ ; (2) if we find some  $te_i \leq t_0 < ts_{i+1}$ , we answer  $t^* := ts_{i+1}$ .

We can avoid recomputing the same answers for all the time instants  $t$  of an interval: Rather than returning each time instant, we return the range  $[t_0, te_i]$  in case (1) above, and  $[ts_{i+1}, te_{i+1}]$  in case (2). We can then modify the intersection algorithm so that we record the maximum interval starting at  $t^*$  that is included in all the intersected lists. The instantiation of the corresponding time variable is then a whole interval where the instantiated BGP will not change. For a pseudocode see the extended version (Algorithm 3 in Appendix B).

Thus we can return solutions to BGPs with ranges of values for the time-bound variables. This is a form of compacting the output, but we cannot ensure we return it in the optimally compacted form.

## 6 LINEAR SPACE

The base version we have described implements leap in  $O(\log N)$  time and  $O(N \log N)$  space. This space is worrisome for large graphs. We now describe a more sophisticated storage mechanism that achieves linear space with no penalty in time complexity.



**Figure 5:** On the left, the two final levels of an LTJ trie node rooted at  $v$ . In the middle, the BT representation of the corresponding children. Note that we can also regard all the BTs as a single BT on the concatenation of the path labels,  $2 : 5$ ,  $3 : 4$ ,  $3 : 5$ ,  $6 : 4$ ,  $6 : 6$ , and  $6 : 7$ . On the right, a VBT representing two edits: the removal of  $2 : 5$  and the insertion of  $6 : 1$ .

### 6.1 Data Structure

Consider the sequence of updates that occur in the time-level children  $[ts_1, te_1), [ts_2, te_2), \dots$  of a particular LTJ trie node. Instead of creating a sequence of VBTs  $v_1, v_2, \dots$ , we create a *single* BT  $V$  where we insert all the paths corresponding to those updates; each VBT (or BT)  $v_l$  corresponds to a sequence of insertions/deletion of tuples (just insertions in case of BTs). If there are  $b$  levels after the time level, for  $1 \leq b \leq 3$ , each such path is of length  $bl$ . Updates consisting of insertions correspond to a path of length  $bl$ . Updates consisting of deletions are also paths of length  $bl$ , plus a deletion mark. Note this differs from the way we represented deletions in VBTs; see Figure 5. If a given tuple is inserted and deleted several times, its leaf will correspond to several updates.

We will record the timestamps of those updates. If  $V$  represents  $L$  updates, they are numbered 1 to  $L$ . Each node  $[ts_l, te_l)$  stores the value  $p_l \in [1, L]$  for its last update. This means that its BT corresponds to executing the updates  $[1, p_l]$  on an empty trie. We store the timestamps, however, in a way that will enable fast navigation.

Each node  $v$  of  $V$  will conceptually store the subsequence  $v.T$  of  $[1, L]$  corresponding to the updates that occur below  $v$ . The subsequence is  $v.T = \langle 1, 2, \dots, L \rangle$  if  $v$  is the root of  $V$ . If  $v$  is a leaf, then  $v.T$  contains the timestamps where its particular tuple was created or removed. Figure 6 shows an example (see only the top and bottom-left parts for now).

Instead of storing  $v.T[1, L_v]$  explicitly, we will store only a bitvector  $v.B[1, L_v]$ , where  $v.B[i] = 0$  iff  $v.T[i]$  appears below the left child of  $v$ , and  $v.B[i] = 1$  if it appears below the right child. To efficiently support operation leap, we also store a bitvector  $v.E[1, L_v]$ , where  $v.E[i] = 1$  iff at timestamp  $v.T[i]$  there exists some tuple below  $v$  whose last update was an insertion (i.e., the tuple exists at timestamp  $i$ ). The leaves of  $V$  store only their bitvector  $v.E$ , not  $v.B$ . See the bottom part of Figure 6. The left child of the root has  $E[4] = 0$  since the node does not exist at timestamp  $T[4] = 6$ . Its right child has  $B = 0101$  since timestamps  $T[1] = 1$  and  $T[3] = 4$  go to its left child, whereas  $T[2] = 2$  and  $T[4] = 6$  go to the right.

Note that every update induces  $2bl + 1$  bits in  $V$ : one in the bitvectors  $v.B$  of the proper ancestors  $v$  of its leaf and one in the bitvectors  $v.E$  of those nodes and the leaf itself. We can actually avoid storing  $v.E$  at leaves  $v$  because it is always an alternating

sequence of 1s and 0s. Therefore, the total number of bits in  $v.B$  and  $v.E$  bitvectors of  $V$  is  $L \cdot 2bl = O(L \log N)$ . The machine word must hold  $\Omega(\log N)$  bits if it can address  $\Theta(N)$  tuples in constant time; therefore this number of bits amounts to  $O(L)$  words of space. Since the sum of the lengths  $L$  of all the nodes below the time level amounts to the  $2N$  updates in the graph, those  $O(L)$  words amount to  $O(N)$  over the whole LTJ trie.

We still have  $O(L \log N)$  space to store the topology of  $V$ , however, as it stores  $L$  paths of length  $bl = O(\log N)$ . We avoid the need to store those pointers by concatenating all the bitvectors  $v.B$  (and, similarly,  $v.E$ ) levelwise, left to right. Note that the total length of the bitvectors  $v.B$  or  $v.E$  in a given level is always  $L$ , so we have  $bl$  bitvectors  $B_d[1, L]$ , for  $d \in [0, bl - 1]$  that can be stored as a large concatenated bitvector of length  $Lbl$  (similarly,  $E_d$  for  $d \in [0, bl]$ , where the level  $d = bl$  can be made implicit, as explained). Those bitvectors are shown at the bottom-right of Figure 6.

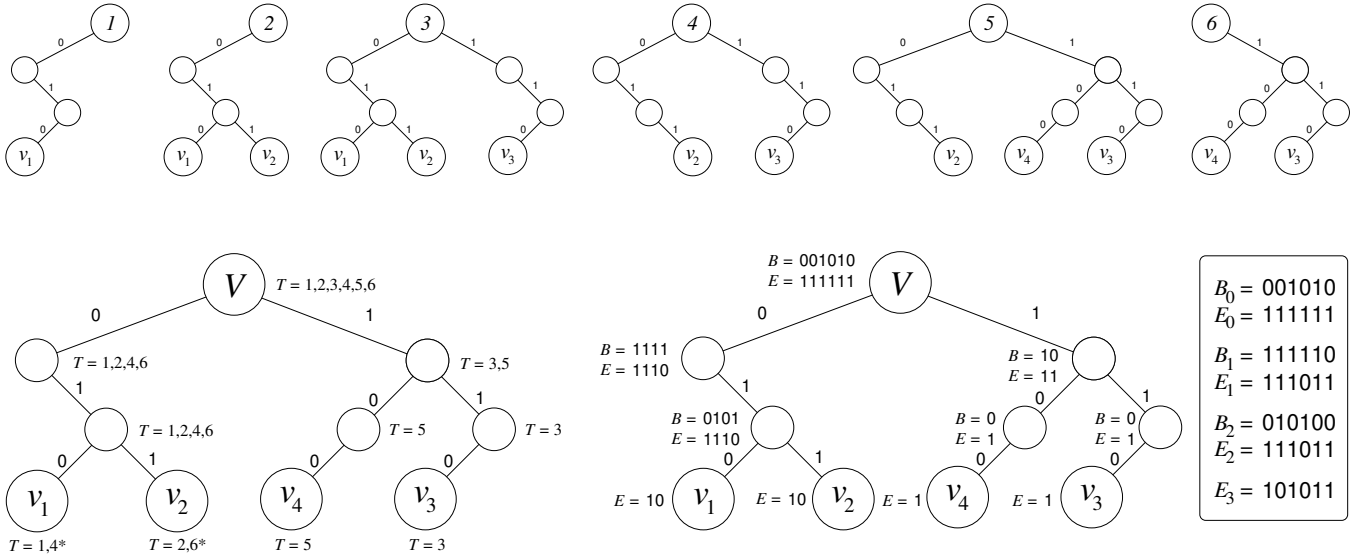
### 6.2 Navigation

To navigate  $V$  without pointers, we need to know the ranges  $B_d[s_v, e_v]$  and  $E_d[s_v, e_v]$  where the bitvectors  $B_v[1, L_v]$  and  $E_v[1, L_v]$  are stored, for any node  $v$  of depth  $d$ . For the root, these are just  $B_0[1, L]$  and  $E_0[1, L]$ . For the rest, we make use of the function  $rank(B, i, j)$ , which counts the number of 1s in  $B[i, j]$ . This operation can be computed in constant time by storing just  $o(|B|)$  additional bits [11, 29]. Assume  $B_d[s_v, e_v]$  is the area of  $B_d$  corresponding to  $v.B$ . Then, letting  $r := rank(B_d, s_v, e_v)$ , the area for the left child of  $v.B$  is  $B_{d+1}[s_v, e_v - r]$ , and the area for its right child is  $B_{d+1}[e_v - r + 1, e_v]$ . We know we have reached a null pointer  $v$  in  $V$  when  $s_v > e_v$ .

Let us start at the root in Figure 6, with  $B_0[1, 6] = 001010$ . To go to its left child, we compute  $rank(B_0, 1, 6) = 2$ ; thus its left child corresponds to  $B_1[1, 4] = 1111$ . If we want to go left again, we compute  $rank(B_1, 1, 4) = 4$ , and find that its left child is null because it corresponds to  $B_2[1, 0]$ .

Up to here, our structure is a variant of the wavelet matrix [12], but the bitvectors  $E_d$  extend it so that we can traverse  $V$  as if it were the VBT at some timestamp  $p_l \in [1, L]$ . We start at the root of  $V$  (i.e.,  $d = 0$  and  $[s_v, e_v] := [1, L]$ ) with the local offset  $p := p_l - s_v$  for  $p_l$ . If  $E_d[s_v + p] = 0$ , then the VBT node was null at this time instant. Otherwise, we can go left or right. The





**Figure 6:** On the top, the BTs of six consecutive timestamps, corresponding to inserting  $v_1$  with value  $010_2 = 2$ , then  $v_2$  with value  $011_2 = 3$ , and  $v_3$  with value  $110_2 = 6$ . The leaf  $v_1$  is then removed,  $v_4$  is inserted with value  $100_2 = 4$ , and finally  $v_2$  is removed. On the bottom left, our conceptual trie  $V$  representing all the events, with the  $v.T$  sequences of timestamps besides each node  $v$ ; the deletions are marked with a \* at the leaves. On the right, the actual compact representation of  $V$ , first still with the tree topology and then just like the bitvectors  $B_d$  and  $E_d$ . Bitvector  $E_3$  can be made virtual, as explained.

timestamp  $p$  becomes  $p - \text{rank}(B_d, s_v, s_v + p)$  on the left child and  $\text{rank}(B_d, s_v, s_v + p) - 1$  on the right child. Note that  $p$  cannot become negative, since that would mean that there have been no updates on the subtree of  $V$  since the beginning, and thus the node would be null at timestamp  $p$  (which we detect before entering the node).

Assume we want to go to the left child of the root at timestamp  $p_l = 6$  in Figure 6. Although  $V$  has such a node, which is  $B_1[1, 4]$ , we realize that it did not exist at timestamp 6 because  $E_1[1 + 3] = 0$ . This position 3 is where we map the relative value  $p = p_l - 1 = 5$  at the root to relative value  $p - \text{rank}(B_0, 1, 6) = 3$  at its left child.

With those operations, we can traverse any VBT top-down in constant time per movement to children, while the total space used by  $V$  is  $O(L)$  and the total space is  $O(N)$  per LTJ trie.

To carry out the operation leap on this representation, recall that leap is applied on an LTJ trie node  $[ts_l, te_l]$  of the time level, and aims to traverse its VBT (or BT) from its root  $v$ . While previously we describe this on a VBT of height  $\ell$ , as we always perform leap on a single attribute. We have now concatenated  $b$  attributes in VBTs of height  $b\ell$ . However, because we can start at any node whose depth is a multiple of  $\ell$ , the procedure remains unchanged over blocks of  $\ell$  elements. When run on  $V$ , we are only interested in the timestamps  $[1, p_l]$ , which represent the VBT of  $[ts_l, te_l]$ . We use  $v.E$  as explained to determine if a node is null at timestamp  $p_l$ . Pseudocode in the extended version (Algorithm 4). This completes the proof of Theorem 1.

## 7 QUERYING FOR POINTS AND INTERVALS

Definition 7 introduces several labeled graphs derived from a temporal graph  $G$ , in which it is of interest to answer standard BGPs.

We now show how our linear-space data structure on  $G$  can be used to answer BGPs over these graphs. We start with point-in-time and point-in-interval queries, corresponding to querying graphs  $G_t$  and  $G_{[t_1, t_2]}$ , where  $t$ ,  $t_1$ , and  $t_2$  are given at query time (Theorem 2). We finish with  $G_{[t_1, t_2]}$  and queries with duration. (Theorem 3).

### 7.1 Point-in queries

To answer a query  $Q$  over  $G_t$ , we transform it into a temporal BGP by adding the constant  $t$  as the fourth component in all the triples. Interestingly, if we force the variable order to descend first by the attribute  $\tau = t$  we can show that the resulting algorithm takes exactly the same amount of steps that the normal LTJ algorithm on  $Q$  would take, immediately giving us optimality over  $G_t$ . This proves the first part of Thm. 2.

For the case of  $G_{[t_1, t_2]}$ , we must extend the navigation of  $G_t$  using VBTs, described in Section 6.2, to intervals  $[t_1, t_2]$ . The key idea is that if  $t_1$  and  $t_2$  are represented by  $p_1$  and  $p_2 + 1$  at some node  $[s_v, e_v]$ , then  $\text{rank}(E_d, s_v + p_1, s_v + p_2) = 0$  iff the node  $s_v$  did not exist along the whole period between the local offsets  $p_1$  and  $p_2$ , that is, it did not exist in  $G_{[t_1, t_2]}$ . Hence, we can quickly find the time intervals that overlap or are contained in  $[t_1, t_2]$ , and continue the LTJ algorithm while keeping track of these intervals. This yields the second part of Thm. 2; further details in the extended version.

We note that the so-called “join-first” strategy, which solves  $Q$  on the labeled graph  $G_{\text{all}} = G_{[1, T]}$  of all the tuples that ever existed and then filters the results by time, can offer the AGM bound only on  $|G_{\text{all}}|$ , whereas our result bounded by  $|G_{[t_1, t_2]}|$  is the best one can hope for: the algorithm is wco on the labeled graph that has exactly the triples we want to consider.

Given a single triple pattern  $(s, p, o)$ , our algorithm lists all matches that existed during the interval  $[t_1, t_2]$  in instance-optimal time:  $O(\log N)$  per reported triple. We can also *list* all the maximal intervals where each such match existed during  $[t_1, t_2]$ , each in  $O(\log N)$  time. This also permits tracking the differences between  $G_{t_1-1}$  and  $G_{t_2-1}$ : an important operation on versioned graphs. Neither “join-first” nor “time-first” strategies can handle these queries near-optimally. Full proof in the extended version (Appendix C).

**THEOREM 4.** *Let  $G$  be a temporal graph with  $N$  tuples. Then, there is a data structure using  $O(N)$  space that can report all the occurrences of a single-triple-pattern query  $Q = \{(s, p, o)\}$  in the labeled graph  $G_{[t_1, t_2]}$ , for any time interval  $[t_1, t_2]$  given with  $Q$ , in time  $O((1 + \text{occ}) \log N)$ . It can also list each maximal time interval where each occurrence appears in time  $O(\log N)$ .*

## 7.2 Queries with duration

In some cases we are interested in requiring that the solutions to the queries last for some time. We consider two cases of such queries. The first is to establish a time interval and look for solutions that always hold during this interval. According to our definitions, this amounts to running the query over  $G_{[t_1, t_2]}$ ; recall Def. 7. A second case does not fix the exact times, but sets a minimum duration  $\delta$  along which the reported solutions must hold [21].

In order to query  $G_{[t_1, t_2]}$  in optimal time, we can reuse some of the ideas used to query  $G_{[t_1, t_2]}$ . Indeed, we have that  $\text{rank}(E_d, s_v + p_1, s_v + p_2) = p_2 - p_1 + 1$  iff the node  $s_v$  exists throughout the whole period between the local offsets  $p_1$  and  $p_2$ . Therefore, we can use Algorithm 5 with the only change that we require that all the bits in the area of  $E_d$  are 1s, not just one of them. This may not be optimal in the size of  $G_{[t_1, t_2]}$ , because we may be stuck exploring a BT node that existed all along  $[t_1, t_2]$ , while none of its children did. However, the algorithm is still wco with respect to the size of  $G_t$  for any  $t \in [t_1, t_2]$ : if the subtree of a node  $v$  at depth  $d$  does not exist in  $G_t$ , then the corresponding bit  $E_d$  will be zero and the algorithm will not attempt to explore it. Instead of satisfying the AGM bound on  $|G_{[t_1, t_2]}| = |\cap_{t \in [t_1, t_2]} G_t|$ , we satisfy it in terms of  $\min_{t \in [t_1, t_2]} |G_t|$ . This proves the first part of Thm. 3.

For the case where we look for solutions of a query  $Q$  that hold over a minimum duration  $\delta$ , we first solve the query on  $G$ , adding the same variable  $t$  as the fourth component of all the triple patterns, and use the LTJ tries where the time component  $\tau$  is at the end, as in join-first approaches. For the time level we use a more sophisticated data structure, based on geometric grids, that allows us to quickly discard time intervals smaller than  $\delta$  while at the same time storing the children of each LTJ trie node  $v$  in an efficient way; see the extended version (Appendix D). This is the second part of Thm. 3.

## 8 IMPLEMENTATION

Although our data structure uses linear space, a naive implementation would need to store  $4! = 24$  tries with all the possible orderings of  $\{s, p, o, t\}$ . Since each trie has 4 levels and all the tuples are mentioned once in every level before the time level and twice since the time level, this amounts to  $156N$  trie nodes for a temporal graph of  $N$  tuples. In order to reduce this overhead we make use of *trie switching* [6] and *partial tries*. Trie switching avoids representing some of the  $4!$  orders as follows: assume we need to descend with

order  $\text{psot}$ . We do not need that particular trie if we have instead those for  $\text{psot}$  and  $\text{sptot}$ : once we have descended by  $p$  and  $s$  in the trie for  $\text{psot}$ , we can reenter the trie  $\text{sptot}$  from the root, using the current values of  $s$  and  $p$ , in order to continue descending by  $t$  and  $o$ . In the extended version (Appendix E) we show a meta-trie that simulates LTJ on any instantiation order via trie switching and uses just  $76N$  nodes. Since we need  $5N$  words to store  $N$  tuples  $(s, p, o, [ti, tf])$  in raw form, this amounts to 15.2 times the space needed to store the raw data. This is in line with the space blowup of well-known implementations of labeled graph databases: MillenniumDB [37], a recent wco index for (non-temporal) labeled graphs, uses 13 times the space needed to store the raw data, for example.

To further reduce space, we use a recent compact trie representation [5] for the levels up to the time component; subsequent ones are implemented as described in Section 6.

## 9 EXPERIMENTS

We now evaluate the performance of our techniques over temporal graphs. We focus on three questions. The first evaluates whether the asymptotic guarantees of our index result in an implementation that is efficient in practice, rather than merely optimal in theory.

**Q1:** How do the theoretical time and space bounds of our index structure translate into practical performance?

An aspect overlooked in the theory of worst-case optimal algorithms is variable binding order, which does not affect theoretical bounds, but does affect performance in practice. Hence we ask:

**Q2:** How do different variable elimination orders affect the performance of our algorithm? Which ordering strategy provides the best results in terms of performance?

A key advantage of our index structure is its support for variable orderings that bind time variables across graph variables. Our third question evaluates the practical impact of this flexibility.

**Q3:** Can we design efficient variable ordering strategies beyond time-first and join-first?

### 9.1 Experimental Setup

**Benchmark.** We perform experiments over the Wikidata knowledge graph [36]. We choose Wikidata as it provides a (1) large-scale, (2) real-world, (3) diverse knowledge graph featuring (4) temporal annotations, further providing a comprehensive log of (5) real-world queries [27]. As per many real-world temporal graphs, some relations include temporal annotations, while others are asserted without temporal qualification (considered universally valid).

We extract the temporal graph from Wikidata considering all  $N = 844,172,299$  triples consisting of items and properties. We extract temporal qualifiers on Wikidata statements, where available, for start time ( $ti$ ), end time ( $tf$ ), and point in time ( $ti$  and  $tf$ ). Since temporal values have different levels of precision, we map them to a uniform precision of date (i.e., day-level precision). In the resulting temporal graph, 15,943,638 triples have non-trivial temporal annotations. Our experiments consider two graphs: the *full graph* considers all 844,172,299 triples including both universally-valid and temporally-restricted relations, while the *transient graph* only considers the 15,943,638 triples with non-trivial temporal annotations. Both graphs include  $|\mathcal{T}_G| = 155,956$  unique time instances.

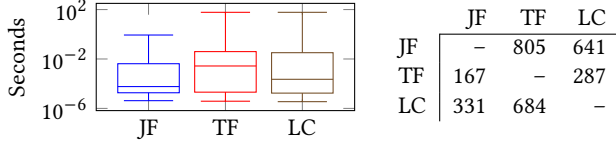


Figure 7: Left: Boxplots of execution times in seconds on the full graph. Right: number of queries for which the approach in the row gives a lower runtime than that in the column.

We extract 1000 BGPs from the real-world query logs published for Wikidata [27]; more details are given in the extended version (Appendix F). We filter queries whose constants are not contained in the full and transient graphs, resulting in 972 and 720 BGPs, respectively. We run queries with a result limit of 1,000 and a timeout of 60 seconds. Our benchmark and code are available on Github.

*Machine.* We ran our experiments on a 16-core Intel Xeon Silver 4110 with Debian 5.10.127-2, clocked at 2.1 GHz, with 768 GB of RAM and 10 MB cache. Our C++ code was compiled with -Ofast.

*Indexing.* Our index built on the full graph uses 240.7 bytes per edge, or 60.2 32-bit words. This is 12 times the space needed to store the edges in plain form (as five 32-bit integers). For comparison, a standard index on non-temporal graphs uses 157 bytes per edge [5]. Our space is better than expected because higher trie levels have fewer than  $N$  elements. From the total space, 84% is used by the 6 tries starting with the time component, with  $\text{TSPO}$ ,  $\text{TPoS}$ , and  $\text{rSPo}$  being the largest tries (see the extended version, Appendix F).

The indexing time for the 18 partial tries on the full graph was 9 hours: 38 microseconds per edge. It grows linearly with the data size (which we verified by indexing increasing subsets of the collection). About 9% of this time was spent in building the 12 regular tries; the rest is used to build the 6 tries starting with the time component. This is because building the linear-space data structure we designed in Section 6 requires sorting the data  $\lceil \log_2 |\mathcal{U}_G| \rceil = 28$  times per level, not just once as the standard tries (on the other hand, those are faster radix-sorts). Our construction was essentially in-place, using no more space than the final index.

*Variable elimination.* We compare the following strategies:

- JF** (Join-First): The order is decided first as if the query did not have a time component, hence doing a standard LTJ join, as in e.g. [6, 20], and then we process all time variables.
- TF** (Time-first): First we process all time variables, and then proceed with a non-temporal LTJ ordering.
- LC** (Least children): A general strategy used in LTJ [20] in which the next variable is the one whose trie nodes have the least amount of children, while requiring that the next variable to bind is connected to a previously bound one, when possible.

## 9.2 Results

Figure 7 provides boxplots of runtimes for all queries evaluated with the three approaches on the full graph, along with the number of queries for which one approach is faster than another. Globally, the **TF** strategy seems to be the weakest, and **JF** the strongest. However, by not being required to bind the temporal variables at

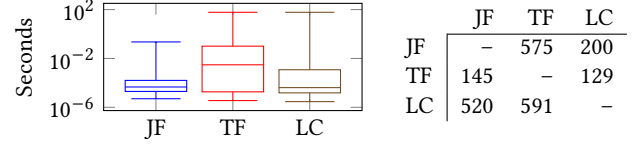


Figure 8: Same experiments as those in Figure 7, now considering the transient graph.

the beginning (per **TF**) nor at the end (**JF**), the **LC** strategy yields better times for over a third of the queries.

The results for **LC** are not better overall because a relatively small fraction of our graph contains temporal information, while other edges are valid all the time, making the time variable less selective and making it more convenient to eliminate it at the end (per **JF**). To control for this effect, Figure 8 presents the corresponding results for the transient graph, showing that **LC** is now the best strategy on more than 70% of the queries, the best in the median case, though still outperformed by **JF** at higher percentiles.

## 9.3 Discussion

The median times of all strategies for our benchmark support deploying temporal queries in practice. Using our index structure we have shown it is possible to answer realistic queries within a few milliseconds in most cases, and with little space overhead: we need 240.7 bytes per edge, which is less than double that needed for a non-temporal graph. Hence, we answer **Q1** in a positive way.

Regarding **Q2**, we see a substantial impact of the type of strategy used. We see that **TF** is generally the worst, with **JF** outperforming **LC** on the full graph in the median case, and **LC** likewise outperforming **JF** on the transient graph. We conclude that evaluating temporal variables last is a reasonable heuristic, but particularly in cases where the temporal validity of triples in the graph is more selective (per our transient graph), it is often highly beneficial to allow temporal variables to be eliminated earlier (per **LC**).

Furthermore, the matrices of Figures 7 and 8 show that being able to evaluate queries in different orderings matters, as for some queries **LC** yields a substantial speedup: **LC** is faster than **JF** on 35% of the queries in our benchmark, and achieves at least a 10% reduction in over 20% of queries. When most edges have temporal information, **LC** outperforms **JF** and **TF** over 70% of the time. This answers **Q3** positively: while **JF** is more robust overall, there is room for designing better heuristics along the lines of **LC**.

## 9.4 Comparison with previous work

Though we have not found available implementations to directly compare with our system, our query language supports the queries studied by Zhu et al. [39], who give sufficient details to perform similar experiments—though no public code. They study, on various datasets, BGPs with fixed topologies (stars, chains, cycles, cliques of various sizes) where all edges must exist at the same time within a given window  $[t_s, t_e]$ . This is equivalent, in our language, to adding a time variable  $t$  to all their BGP triples, and adding the temporal constraints  $t_s \leq t$  and  $t \leq t_e$ . For each topology, they average the time over 100 queries where all nodes are variables and edges are

| System       | 3-star | 4-star | 5-star | 3-chain | 4-chain | 5-chain | 3-circle | 4-circle | 5-circle | diamond | 4-clique | 5-clique |
|--------------|--------|--------|--------|---------|---------|---------|----------|----------|----------|---------|----------|----------|
| TSRJoin [39] | 50     | 50     | 50     | 50      | 50      | 70      | 150      | 150      | 70       | 100     | 300      | 700      |
| Time [39]    | 2,000  | 2,000  | 2,000  | 2,000   | 2,000   | 5,000   | 5,000    | 6,000    | 7,000    | 5,000   | 20,000   | 40,000   |
| Ours         | 12.85  | 23.93  | 18.44  | 4.30    | 6.04    | 4.52    | 4.36     | 4.39     | 4.98     | 8.60    | 4.39     | 6.58     |

**Table 1: Average time in milliseconds to solve the different shapes, with a time window of 10% of the domain and limiting results to 100,000. Times from Zhu et al. [39] are approximated from their plots.**

| Query    | Time window selectivity |      |      |       |       | Output size limit |        |        |        |        |
|----------|-------------------------|------|------|-------|-------|-------------------|--------|--------|--------|--------|
|          | 0.01%                   | 0.1% | 1%   | 10%   | 20%   | $10^3$            | $10^4$ | $10^5$ | $10^6$ | $10^7$ |
| 3-circle | 0.12                    | 0.26 | 0.80 | 4.36  | 7.23  | 4.06              | 4.08   | 4.36   | 4.06   | 4.08   |
| 4-star   | 0.29                    | 0.70 | 4.22 | 23.93 | 52.10 | 15.82             | 23.88  | 23.93  | 24.07  | 24.16  |
| 5-clique | 0.14                    | 0.30 | 1.13 | 6.58  | 10.50 | 6.60              | 6.61   | 6.58   | 6.59   | 6.61   |

**Table 2: Average time in milliseconds to solve three different shapes with our system, first with a time window from 0.01% to 20% of the domain and limiting results to 100,000, and second with a time window of 10% and limiting the results to  $10^3$  to  $10^7$ .**

(valid) random constants; they also use random time windows that go from 0.01% to 20% of the time domain. Further, they consider queries that deliver from 1 to at most  $10^3, \dots, 10^7$  results.

From their datasets, we could reproduce **Divvy** with reasonable accuracy, which records 20 million bike trips: the source and target locations become the subject and object, and the type of bike is taken as the predicate (there are many fine-grained decisions they do not explain, but we believe the results should be comparable). The graph has  $N = 21,243,344$  edges,  $|\mathcal{U}_G| = 7,158$  identifiers for nodes and labels, and almost as many timestamps as edges,  $|\mathcal{T}_G| = 20,673,270$ . Their TSRJoin index (the one that performs best) takes 483 bytes per tuple and is built in 133 seconds in their machine (which is clocked at 2.9 GHz, but its architecture is older). Our index, instead, takes 190 bytes per tuple (i.e., 40% of their space) and is built in 531 seconds (i.e., 4 times slower).

Table 1 compares our query time with those of Zhu et al. [39] with a time window of 10% of the domain and limiting the results to 100,000, which is the only configuration where we can compare all query shapes. We show their best time (TSRJoin), and the next-best (Time, a time-first approach). It can be seen that our approach, which in this case works best with time-first, is 2–100 times faster than TSRJoin and 3–4 orders of magnitude faster than Time. While the 2–4 speedup factors on star shapes can be attributed to our more modern machine<sup>1</sup>, we are 1–2 orders of magnitude faster on the other shapes. We conjecture that this is mainly due to our ability to choose arbitrary VEOs for the other variables: we outperform TSRJoin more sharply on the queries where it is not trivial to find the best VEO. On the star queries, instead, there is only one join variable and the time is dominated by the large output size.

Table 2 shows, on a few “easy” and a “hard” query shapes, how our query times evolve as we vary the selectivity of the window and the limit on the output size. The time window sensitivity has a significant impact on query time, showing that our index filters effectively using the time component. The output size limit has, instead, little impact: only some on the 4-star, whose time is largely related to the number of outputs produced.

<sup>1</sup>According to Gemini, our hardware should be from 25%–35% faster for pure-CPU calculations, to 3 times faster for computations bounded by RAM transfers.

## 10 CONCLUSIONS

We have described a data structure that enables processing temporal BGPs in wco time, and provides further guarantees for other key applications, such as point-in-time queries or queries with duration.

One interesting challenge for future work is to obtain beyond-wco query times. For example, we can easily obtain time related to the generalized hypertree width of the query [1], which is never larger than its fractional hypertree width (the AGM bound). This can be done by optimally partitioning the query into a tree of cyclic queries, using our wco algorithm to solve each cyclic component, and then applying Yannakakis’ algorithm [38] to solve the resulting acyclic join of intermediate results. A problem is that those intermediate results might be obtained with the bindings of the time variables in non-compacted form (akin to representing  $\hat{G}$  instead of  $G$ ), and then they might require a lot of space. It would be interesting to represent the intermediate results using time intervals, within minimum space; adapting Yannakakis’ semijoins to this representation does not seem to be problematic. It is possible to find the optimal set of hyperrectangles covering all the instantiations of the temporal variables, in optimal space and time, if the VEO binds them at the end, as they are then output in lexicographical order for each instantiation of the other variables. It does not seem possible to achieve the same in the general case, however.

A related challenge for future research is to choose the most efficient variable order in more cases. As seen in the matrices of Figures 7 and 8, which approach works best depends on the query. The least children (LC) strategy—akin to greedy optimization on cardinality estimates—works well in many cases, but there is room for better heuristics, especially for universally-true triples.

Another line of future work is to extend the language of the temporal clauses. For example, we may include relations of the form  $w_1 < w_2$  or timestamp arithmetic like  $w_1 \leq w_2 + 3$  if  $\mathcal{T}$  is numeric, which cannot be handled without representing the original time domain  $\mathcal{T}$  (rather than  $\mathcal{T}_G$ ). In this case, the point graph  $\hat{G}$  is only bounded by  $N \cdot |\mathcal{T}|$ , which can even be infinite, and worst-case optimality must be reconsidered. Still, our structures would work without many changes. It is still crucial to always bind temporal variables to ranges, as discussed in Section 5.3.

## REFERENCES

- [1] Christopher R Aberger, Andrew Lamb, Susan Tu, Andres Nötzli, Kunle Olukotun, and Christopher Ré. Emptyheaded: A relational engine for graph processing. *ACM Transactions on Database Systems (TODS)*, 42(4):1–44, 2017.
- [2] Mahmoud Abo Khamis, George Chichirim, Antonia Kormpa, and Dan Olteanu. The complexity of boolean conjunctive queries with intersection joins. In *Proceedings of the 41st ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems*, pages 53–65, 2022.
- [3] Avishek Anand, Srikanta Bedathur, Klaus Berberich, and Ralf Schenkel. Index maintenance for time-travel text search. In *Proceedings of the 35th international ACM SIGIR conference on Research and development in Information Retrieval*, pages 235–244, 2012.
- [4] Renzo Angles, Marcelo Arenas, Pablo Barceló, Aidan Hogan, Juan Reutter, and Domagoj Vrgoč. Foundations of modern query languages for graph databases. *ACM Computing Surveys (CSUR)*, 50(5):1–40, 2017.
- [5] Diego Arroyuelo, Daniela Campos, Adrián Gómez-Brandón, Yuval Linker, Gonzalo Navarro, Carlos Rojas, and Domagoj Vrgoč. CompactLTJ: Space & time efficient Leapfrog Triejoin on graph databases. *The Very Large Databases Journal*, 34:article 67, 2025.
- [6] Diego Arroyuelo, Adrián Gómez-Brandón, Aidan Hogan, Gonzalo Navarro, Juan L. Reutter, Javier Rojas-Ledesma, and Adriá Soto. The Ring: Worst-case optimal joins in graph databases using (almost) no extra space. *ACM Transactions on Database Systems*, 29(2):article 5, 2024.
- [7] Albert Atserias, Martin Grohe, and Daniel Marx. Size bounds and query plans for relational joins. *SIAM Journal on Computing*, 42(4):1737–1767, 2013.
- [8] Klaus Berberich, Srikanta Bedathur, Thomas Neumann, and Gerhard Weikum. A time machine for text search. In *Proceedings of the 30th annual international ACM SIGIR conference on Research and development in information retrieval*, pages 519–526, 2007.
- [9] Michael H Böhlen, Christian S Jensen, and Richard Thomas Snodgrass. Temporal statement modifiers. *ACM Transactions on Database Systems (TODS)*, 25(4):407–456, 2000.
- [10] Timothy M. Chan, Kasper G. Larsen, and Mihai Pătraşcu. Orthogonal range searching on the RAM, revisited. In *Proc. 27th ACM Symposium on Computational Geometry (SoCG)*, pages 1–10, 2011.
- [11] David R. Clark. *Compact PAT Trees*. PhD thesis, University of Waterloo, Canada, 1996.
- [12] Francisco Claude, Gonzalo Navarro, and Alberto Ordóñez. The wavelet matrix: An efficient wavelet tree for large alphabets. *Information Systems*, 47:15–32, 2015.
- [13] Albert Croker and James Clifford. On completeness of historical relational data models. 1989.
- [14] Ignacio Cuevas and Aidan Hogan. Versioned queries over RDF archives: All you need is SPARQL? In *MEPDAW@ ISWC*, pages 43–52, 2020.
- [15] Anton Dignös, Michael H Böhlen, and Johann Gamper. Temporal alignment. In *Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data*, pages 433–444, 2012.
- [16] Nadime Francis, Amélie Gheerbrant, Paolo Guagliardo, Leonid Libkin, Victor Marsault, Wim Martens, Filip Murlak, Liat Peterfreund, Alexandra Rogova, and Domagoj Vrgoč. A researcher’s digest of GQL. In *The 26th International Conference on Database Theory, 2023*, pages 1–1. Schloss Dagstuhl-Leibniz-Zentrum für Informatik, 2023.
- [17] Yunjun Gao, Tianming Zhang, Linshan Qiu, Qingyuan Linghu, and Gang Chen. Time-respecting flow graph pattern matching on temporal graphs. *IEEE Transactions on Knowledge and Data Engineering*, 33(10):3453–3467, 2020.
- [18] Fabio Grandi et al. Multi-temporal RDF ontology versioning. In *IWOD@ ISWC*, 2009.
- [19] Fabio Grandi et al. T-SPARQL: A TSQ2-like temporal query language for RDF. In *ADBIS (local proceedings)*, volume 639, pages 21–30, 2010.
- [20] Aidan Hogan, Cristian Riveros, Carlos Rojas, and Adrián Soto. A worst-case optimal join algorithm for SPARQL. In *Proc. 18th International Semantic Web Conference (ISWC)*, pages 258–275, 2019.
- [21] Xiao Hu, Stavros Sintos, Junyang Gao, Pankaj K. Agarwal, and Jun Yang. Computing complex temporal join queries efficiently. In *Proc. International Conference on Management of Data (SIGMOD)*, pages 2076–2090, 2022.
- [22] Chengying Huan, Heng Zhang, Yongchao Liu, Likang Chen, Xuran Wang, Yongchun Jiang, Shaonan Ma, and Yanjun Wu. TeMatch: A fast temporal subgraph matching framework with temporal-aware subgraph matching algorithms. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*, pages 1029–1042. IEEE, 2025.
- [23] Guy Jacobson. Space-efficient static trees and graphs. In *Proc. 30th IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 549–554, 1989.
- [24] Manolis Koubarakis and Kostis Kyzirakos. Modeling and querying metadata in the semantic sensor web: The model stRDF and the query language stSPARQL. In *Extended Semantic Web Conference*, pages 425–439. Springer, 2010.
- [25] Krishna Kulkarni and Jan-Eike Michels. Temporal features in SQL: 2011. *ACM Sigmod Record*, 41(3):34–43, 2012.
- [26] Faming Li, Zhaonian Zou, and Jianzhong Li. Durable subgraph matching on temporal graphs. *IEEE Transactions on Knowledge and Data Engineering*, 35(5):4713–4726, 2022.
- [27] Stanislav Malyshev, Markus Krötzsch, Larry González, Julius Gonsior, and Adrian Bielefeldt. Getting the Most Out of Wikidata: Semantic Technology Usage in Wikipedia’s Knowledge Graph. In *17th International Semantic Web Conference*, volume 11137, pages 376–394. Springer, 2018.
- [28] Vera Zaychik Moffitt and Julia Stoyanovich. Temporal graph algebra. In *Proceedings of The 16th International Symposium on Database Programming Languages*, pages 1–12, 2017.
- [29] J. Ian Munro. Tables. In *Proc. 16th Conference on Foundations of Software Technology and Theoretical Computer Science (FSTTCS)*, LNCS 1180, pages 37–42, 1996.
- [30] Ashwin Paranjape, Austin R Benson, and Jure Leskovec. Motifs in temporal networks. In *Proceedings of the 10th ACM International Conference on Web Search and Data Mining*, pages 601–610, 2017.
- [31] Olivier Pelgrin, Ruben Taelman, Luis Galárraga, and Katja Hose. GLENDa: querying RDF archives with full SPARQL. In *European Semantic Web Conference*, pages 75–80. Springer, 2023.
- [32] Matthew Perry, Prateek Jain, and Amit P Sheth. SPARQL-st: Extending SPARQL to support spatiotemporal queries. In *Geospatial semantics and the semantic web: foundations, algorithms, and applications*, pages 61–86. Springer, 2011.
- [33] Konstantinos Semertzidis and Evaggelia Pitoura. Top-k durable graph pattern queries on temporal graphs. *IEEE Transactions on Knowledge and Data Engineering*, 31(1):181–194, 2018.
- [34] Richard Snodgrass. The temporal query language TQuel. *ACM Transactions on Database Systems (TODS)*, 12(2):247–298, 1987.
- [35] Todd L. Veldhuizen. Triejoin: A simple, worst-case optimal join algorithm. In *Proc. International Conference on Database Theory (ICDT)*, pages 96–106, 2014.
- [36] Denny Vrandečić and Markus Krötzsch. Wikidata: a free collaborative knowledgebase. *Commun. ACM*, 57(10):78–85, 2014.
- [37] Domagoj Vrgoč, Carlos Rojas, Renzo Angles, Marcelo Arenas, Diego Arroyuelo, Carlos Buil-Aranda, Aidan Hogan, Gonzalo Navarro, Cristian Riveros, and Juan Romero. MillenniumDB: An open-source graph database system. *Data Intelligence*, 5(3):560–610, 2023.
- [38] M. Yannakakis. Algorithms for acyclic database schemes. In *Proc. 7th International Conference on Very Large Databases (VLDB)*, pages 82–94, 1981.
- [39] Kaijie Zhu, George Fletcher, and Nikolay Yakovets. Leveraging Temporal and Topological Selectivities in Temporal-clique Subgraph Query Processing. In *37th IEEE International Conference on Data Engineering, ICDE 2021, Chania, Greece, April 19-22, 2021*, pages 672–683. IEEE, 2021.

## A ON THE EXPRESSIVE POWER OF TBGPs

Let us start with the comparison with CQs. To abstract from relational representations of temporal graphs, we consider a relational representation with a single relation  $T$  of arity 5, with the first three positions reserved for triples, the fourth for the start of a time interval, and the fifth for the end of the interval. Then, each temporal graph  $G$  is directly represented as an instance  $I_G$ : for each tuple  $(s, p, o, [ti, tf])$  in  $G$  we add to  $I_G$  the atom  $(s, p, o, ti, tf)$ . In this context, one asks whether every tBGP can be *expressed* as a query in  $I_G$ ; we say that a tBGP  $Q$  can be expressed as a conjunctive query over the relational representation of graphs if one can find a conjunctive query  $Q'$  such that the evaluation of  $Q$  over any temporal graph  $G$  corresponds to the evaluation of  $Q'$  over  $I_G$ .

**PROPOSITION 5.** *There is a family of TBGPs that cannot be expressed as conjunctive queries over the relational representation of graphs, nor as conjunctive queries with inequalities.*

**PROOF.** Assume towards contradiction that every tBGP can be expressed as a conjunctive query over the relational representation of temporal graphs, possibly extended with inequalities.

Consider, for every  $n \geq 1$ , the tBGP  $Q_n$  consisting of a path of length  $n$  required to hold at the same time instant:

$$Q_n = \{(x_0, p, x_1, w), (x_1, p, x_2, w), \dots, (x_{n-1}, p, x_n, w)\}.$$

Intuitively,  $Q_n$  asks for a path of length  $n$  that exists simultaneously at some time point  $w$ .

Suppose there exists a conjunctive query  $Q'_n$  over the relational encoding using the relation  $T(s, p, o, ti, tf)$  that expresses  $Q_n$ .

Let  $H_n$  be a static graph consisting of a path

$$a_0 \xrightarrow{p} a_1 \xrightarrow{p} \dots \xrightarrow{p} a_n.$$

We construct two temporal graphs that have the same frozen structure but different temporal behavior.

**Instance  $G_1$ .** For every edge  $(a_i, p, a_{i+1})$  of  $H_n$ , include the tuple

$$(a_i, p, a_{i+1}, [t_1, t_3])$$

with  $t_0 < t_1 < t_2 < t_3$ . Additionally, add one extra edge

$$(b, p', b', [t_0, t_2]).$$

At time  $t_2$ , all edges of the path are valid simultaneously, hence  $Q_n(G_1)$  contains a solution witnessing time  $w = t_2$ .

Since  $Q'_n$  expresses  $Q_n$ , there must be a homomorphism from the body of  $Q'_n$  into  $I_{G_1}$  mapping the variable corresponding to  $w$  to the value  $t_2$ . Consequently, some atom of  $Q'_n$  must use  $t_2$  as the value obtained from an interval endpoint appearing in position 5 (the interval end), because  $t_2$  only occurs there.

**Instance  $G_2$ .** Now consider the temporal graph obtained by replacing the additional edge with

$$(b, p', b', [t_2, t_4])$$

for  $t_3 < t_4$ , while keeping all path edges unchanged. Again, the frozen graph is identical to  $H_n$ , and now the witnessing time belongs to the start of an interval, namely position 4.

Hence, correctness of  $Q'_n$  implies that there must exist a homomorphism mapping the same time variable to position 4 of some atom.

But we also need to map the atom mapping  $w$  to the fifth position. As the value  $t_2$  is not in the fifth position in any tuple in  $I_{G_2}$ , it follows that there cannot be a homomorphism from  $Q'_n$  to  $I_{G_2}$ , which is a contradiction. Therefore, no conjunctive query (even with inequalities) can express any of the TBGPs  $\{Q_n\}_{n \geq 1}$ .  $\square$

However, TBGPs do coincide with conjunctive queries (with inequalities) when we expand graphs into their point-based representation, which may be of quadratic size with respect to the original temporal graph. While materializing the point-based representation is therefore not feasible in practice, the following result provides a good justification for our language: by focusing on TBGPs we study the basing block of temporal query languages. To make this more precise, consider a different relational representation  $I_G$ , which now stores a tuple  $(s, p, o, t)$  for each  $(s, p, o, t) \in \hat{G}$ . Further, let us say that a tBGP is compatible with a graph  $G$  if the time instants mentioned in  $Q$  are also in  $I_G$ .

**PROPOSITION 6.** *for every tBGP  $Q$  one can construct a conjunctive query  $\hat{Q}$  with inequalities, so that the answer of  $Q$  over a compatible temporal graph  $G$  is the same as the answer of  $\hat{Q}$  over  $\hat{G}$ , and conversely, as long as the inequalities on the conjunctive query are only on variables used in the fourth position of relations.*

**PROOF.** Let  $Q$  be a tBGP. Observe that  $Q$  can be viewed syntactically as a conjunctive query of arity 4, possibly with comparison predicates between variables occurring in the fourth (temporal) position. Hence,  $Q$  can be directly evaluated over the point-based representation  $\hat{G}$  as a standard conjunctive query with inequalities. We show that the answers coincide in both settings.

Throughout the proof we consider only temporal graphs  $G$  for which  $Q$  is *well defined*, that is, all constants appearing in  $Q$  belong to  $\mathcal{U}_G$  and all temporal constants belong to  $I_G$ .

( $\Rightarrow$ ) Let  $f$  be a solution of  $Q$  over the temporal graph  $G$ . Consider any tuple pattern  $(x, y, z, w)$  of  $Q$ . Since  $f$  is a valid assignment, there exists a tuple

$$(f(x), f(y), f(z), [ti, tf]) \in G$$

```

leap( $v, x, h$ )
if  $v = \text{null}$  then return  $+\infty$  ;
if  $v$  is a leaf then return 0 ;
if  $\lfloor x/2^h \rfloor = 1$  then  $x \leftarrow \text{leap}(v.r, x - 2^h, h - 1)$  ;
else
   $x \leftarrow \text{leap}(v.l, x, h - 1)$  ;
  if  $x \neq +\infty$  then return  $x$  ;
   $x \leftarrow \text{leftmost}(v.r, h - 1)$  ;
end
if  $x \neq +\infty$  then  $x \leftarrow x + 2^h$  ;
return  $x$  ;

```

```

leftmost( $v, h$ )
if  $v = \text{null}$  then return  $+\infty$  ;
if  $v$  is a leaf then return 0 ;
if  $v.l \neq \text{null}$  then return leftmost( $v.l, h - 1$ ) ;
return  $2^h + \text{leftmost}(v.r, h - 1)$  ;

```

**Algorithm 2:** The implementation of leap using BTs (or, equivalently, VBTs). The function receives the LTJ trie node  $v$  (which is identified with the BT root), the minimum desired value  $x$ , and the BT height  $h = \ell$ . The left and right children of BT node  $v$  are  $v.l$  and  $v.r$ , respectively.

such that  $ti \leq f(w) < tf$ . By definition of solutions, we may assume that  $f(w) \in \mathcal{T}_G$ . Then, by construction of the point-based representation, the tuple

$$(f(x), f(y), f(z), f(w))$$

belongs to  $\hat{G}$ . Hence every atom of  $Q$  is satisfied in  $\hat{G}$ , and all comparison predicates remain valid. Therefore  $f$  is also a satisfying assignment of  $Q$  evaluated as a conjunctive query over  $\hat{G}$ .

( $\Leftarrow$ ) Conversely, let  $f$  be a satisfying assignment of  $Q$  over  $\hat{G}$ . Since tuples of  $\hat{G}$  have the form  $(s, p, o, t)$  obtained from intervals of  $G$ , for every atom  $(x, y, z, w)$  of  $Q$  such that

$$(f(x), f(y), f(z), f(w)) \in \hat{G},$$

there exists an interval  $[ti, tf)$  with

$$(f(x), f(y), f(z), [ti, tf)) \in G \quad \text{and} \quad ti \leq f(w) < tf.$$

Hence the atom is satisfied under the semantics of tBGPs.

Moreover, because the domain of values and the temporal domain are disjoint, variables occurring in the first three positions cannot be mapped to temporal values and vice versa. Thus comparison predicates involving temporal variables are preserved, and  $f$  is a valid solution of  $Q$  over  $G$ .  $\square$

From Propositions 5 and 6, it follows that none of the techniques developed for relational conjunctive queries can be directly ported into our temporal graph setting, unless we first expand temporal graphs to their point-based representation, this, as we explained, is not feasible to do in practice.

## B PSEUDOCODES

We show pseudocode in Algorithms 2, 3, and 4.

## C QUERYING THE GRAPHS $G_t$ AND $G_{[t_1, t_2]}$ (EXTENDED VERSION)

Definition 7 gives some standard labeled graphs that can be derived from a temporal graph  $G$ , on which it is of interest to answer standard BGPs. We now show how we can use our linear-space data structure on  $G$  to answer BGPs on  $G_t$  and  $G_{[t_1, t_2]}$ , where  $t$ ,  $t_1$ , and  $t_2$  are given together with the query, in wco time with respect to those graphs. The case of  $G_{[t_1, t_2]}$  will be discussed later.

We can answer general BGPs  $Q$  on  $G_t$  by converting them to BGPs on the temporal graph  $G$ , adding the constant  $t$  as the fourth component in all the triples. A particularly interesting result, however, is obtained if we instead descend by the attribute  $\tau = t$  in the 6 LTJ tries that start with attribute  $\tau$ . The subtrees that descend from those nodes correspond to the LTJ tries for  $G_t$ , on the attributes  $s$ ,  $p$ , and  $o$ . For example, if we descend by  $t$  in the trie  $\text{rspo}$ , the subtree of the node we arrive at is isomorphic to the trie  $\text{sro}$  of  $G_t$ . We can then run the normal LTJ

```

 $x \leftarrow 0 ; l \leftarrow 0 ;$ 
while true do
   $[x, y] \leftarrow \text{leap}(v_l, x) ;$ 
   $i \leftarrow (l + 1) \bmod k ;$ 
  while  $i \neq l$  do
     $[x', y'] \leftarrow \text{leap}(v_i, x) ;$ 
    if  $x' > x$  then  $[x, y] \leftarrow [x', y'] ; l \leftarrow i ;$ 
    else  $y \leftarrow \min(y, y') ;$ 
     $i \leftarrow (i + 1) \bmod k ;$ 
  end
  if  $x = +\infty$  then break ;
  report  $[x, y) ;$ 
   $x \leftarrow y ;$ 
end

```

**Algorithm 3:** The modified LTJ iterator to account for intervals when intersecting at the time level. The iterator now returns intervals  $[x, y)$  where all the results will be the same.

```

 $\text{leap}(d, s, e, p, x, h)$ 
if  $s + p \notin [s, e] \vee E_d[s + p] = 0$  then return  $+\infty ;$ 
if  $h = 0$  then return  $0 ;$ 
 $r \leftarrow \text{rank}(B_d, s, e) ;$ 
 $p' \leftarrow \text{rank}(B_d, s, s + p) ;$ 
if  $\lfloor x/2^h \rfloor = 1$  then
   $x \leftarrow \text{leap}(d + 1, e - r + 1, e, p' - 1, x - 2^h, h - 1) ;$ 
  if  $x \neq +\infty$  then return  $x + 2^h ;$ 
  return  $+\infty ;$ 
end
else
   $x \leftarrow \text{leap}(d + 1, s, e - r, p - p', x, h - 1) ;$ 
  if  $x \neq +\infty$  then return  $x ;$ 
  if  $r = 0 \vee E_{d+1}[e - r + p'] = 0$  then return  $+\infty ;$ 
  return  $2^h + \text{leftmost}(d + 1, e - r + 1, e, p' - 1, h - 1) ;$ 
end

```

```

 $\text{leftmost}(d, s, e, p, h)$ 
if  $h = 0$  then return  $0 ;$ 
 $r \leftarrow \text{rank}(B_d, s, e) ;$ 
 $p' \leftarrow \text{rank}(B_d, s, s + p) ;$ 
if  $s \leq e - r \wedge E_{d+1}[s + p - p'] = 1$  then
  return  $\text{leftmost}(d + 1, s, e - r, p - p', h - 1)$ 
end
return  $2^h + \text{leftmost}(d + 1, e - r + 1, e, p' - 1, h - 1) ;$ 

```

**Algorithm 4:** The implementation of leap on our linear-space representation. The function receives the LTJ trie node that corresponds to the range  $[s, e]$  at level  $d$  of our structure, the local timestamp of interest  $p = p_l - s$ , the minimum desired value  $x$ , and the BT height  $h = \ell$ .

algorithm for  $Q$  using those subtrees, exactly as if they were the LTJ tries of  $G_t$ . Since our  $O(N)$  space data structure simulates the operation leap on those subtrees in  $O(\log N)$  time, the result follows easily. This proves the first part of Thm. 2.

To obtain an analogous result for  $G_{[t_1, t_2]}$ , we must extend the navigation of  $G_t$  using VBTs, described in Section 6.2, to intervals  $[t_1, t_2)$ . The key idea is that, if  $t_1$  and  $t_2$  are represented by  $p_1$  and  $p_2 + 1$  at some node  $[s_v, e_v]$ , then  $\text{rank}(E_d, s_v + p_1, s_v + p_2) = 0$  iff the node  $s_v$  did not exist along the whole period between the local offsets  $p_1$  and  $p_2$ , that is, it did not exist in  $G_{[t_1, t_2]}$ .

We start on the time level, where we find the first and last intervals,  $[ts_a, te_a)$  and  $[ts_b, te_b)$ , that overlap or are contained in  $[t_1, t_2)$  (it might be that  $a = b$ , but the answer is empty if no such intervals exist). Let these intervals correspond to timestamps  $p_a$  and  $p_b$  in our



```

leap( $d, s, e, p_1, p_2, x, h$ )
if  $s+p \notin [s, e] \vee \text{rank}(E_d, s+p_1+1, s+p_2)=0$  then return  $+\infty$  ;
if  $h = 0$  then return 0 ;
 $r \leftarrow \text{rank}(B_d, s, e)$  ;
 $p'_1 \leftarrow \text{rank}(B_d, s, s+p_1)$  ;  $p'_2 \leftarrow \text{rank}(B_d, s, s+p_2)$  ;
if  $\lfloor x/2^h \rfloor = 1$  then
     $x \leftarrow \text{leap}(d+1, e-r+1, e, p'_1-1, p'_2-1, x-2^h, h-1)$  ;
    if  $x \neq +\infty$  then return  $x+2^h$  ;
    return  $+\infty$  ;
end
else
     $x \leftarrow \text{leap}(d+1, s, e-r, p_1-p'_1, p_2-p'_2, x, h-1)$  ;
    if  $x \neq +\infty$  then return  $x$  ;
    if  $r = 0 \vee \text{rank}(E_{d+1}, e-r+p'_1+1, e-r+p'_2) = 0$  then return  $+\infty$  ;
    return  $2^h + \text{leftmost}(d+1, e-r+1, e, p'_1-1, p'_2-1, h-1)$  ;
end

leftmost( $d, s, e, p_1, p_2, h$ )
if  $h = 0$  then return 0 ;
 $r \leftarrow \text{rank}(B_d, s, e)$  ;
 $p'_1 \leftarrow \text{rank}(B_d, s, s+p_1)$  ;  $p'_2 \leftarrow \text{rank}(B_d, s, s+p_2)$  ;
if  $s \leq e-r \wedge \text{rank}(E_{d+1}, s+p_1-p'_1+1, s+p_2-p'_2) > 0$  then
    return  $\text{leftmost}(d+1, s, e-r, p_1-p'_1, p_2-p'_2, h-1)$  ;
end
return  $2^h + \text{leftmost}(d+1, e-r+1, e, p'_1-1, p'_2-1, h-1)$  ;

```

**Algorithm 5:** The implementation of leap on our linear-space representation, simulating the graph  $G_{[t_1, t_2]}$ . The function receives the LTJ trie node that corresponds to the range  $[s, e]$  at level  $d$  of our structure, the local timestamps of interest  $p_1 = p_{a-1} - s$  and  $p_2 = p_b - s$ , the minimum desired value  $x$ , and the BT height  $h = \ell$ . We assume that  $\text{rank}(E_d, i, i-1)$  returns  $E_d[i-1]$ , assuming  $E_d[0] = 0$ .

linear-space data structure  $V$  below the time level. We start at depth  $d := 0$ , with interval  $[s_v, e_v] := [1, L]$  at the root  $v$  of  $V$ , and with local offsets  $p_1 := p_{a-1} - s_v$  and  $p_2 := p_b - s_v$ . If  $\text{rank}(E_d, s_v + p_1 + 1, s_v + p_2) = 0$ , then the VBT node was null all along the time interval  $[t_1, t_2]$ ; that is to say, there are no elements of  $G_{[t_1, t_2]}$  descending from  $v$ . Otherwise, we can go left or right. We compute  $r := \text{rank}(B_d, s_v, e_v)$  and  $p'_{1/2} := \text{rank}(B_d, s_v, s_v + p_{1/2})$ . To descend left, we update  $e_v := e_v - r$  and  $p_{1/2} := p_{1/2} - p'_{1/2}$ ; to descend right we update  $s_v := e_v - r + 1$  and  $p_{1/2} := p'_{1/2} - 1$ .

A special case occurs in this process if  $p_1$  and  $p_2$  become equal. This means that the subtree of  $v$  had no update during  $[t_1, t_2]$ , and therefore it has tuples below it during that period iff  $E_d[s_v + p_1] = 1$ , assuming  $E_d[0] = 0$ . Algorithm 5 shows how to modify Algorithm 4 to perform leap on this simulated trie.

Note that if the algorithm arrives at a node  $v$ , there exists at least one edge below  $v$  during  $[t_1, t_2]$ ; therefore we do not spend any time on nodes that do not exist in the LTJ trie of  $G_{[t_1, t_2]}$ . This yields the second part of Thm. 2.

We note that the so-called “join-first” strategy, which solves  $Q$  on the labeled graph  $G_{\text{all}} = G_{[1, T]}$  of all the tuples that ever existed and then filters the results by time, can offer the AGM bound only on  $|G_{\text{all}}|$ , whereas our result bounded by  $|G_{[t_1, t_2]}|$  is the best one can hope for: the algorithm is wco on the labeled graph that has exactly the triples we want to consider.

*Triple patterns.* Consider the case of a BGP formed by a single triple pattern  $(s, p, o)$ . Our algorithm will first descend in the time level, computing  $a$  and  $b$  with a binary search, and then descend in  $V$  by the constants in  $(s, p, o)$ . For each node arrived at, there exists at least one triple in  $G_{[t_1, t_2]}$ , and the algorithm will descend by all left and right branches in the VBTs, reporting all the triples. The total time is instance-optimal,  $O(\log N)$  per reported triple. The existential query, that is, telling whether the triple pattern has a match or not in  $G_{[t_1, t_2]}$ , takes  $O(\log N)$  time. Even this simple query is not handled near-optimally with the “join-first” strategy: there could be many matches for  $(s, p, o)$  out of  $[t_1, t_2]$ . It is also not handled well with “time-first”, which individually considers each time  $t \in [t_1, t_2]$  and solves the query on  $G_t$ : the triple pattern may not appear in many time instants  $t$ .

*Listing time intervals.* It is also possible to list all the maximal intervals where each match  $(s, p, o)$  of the BGP existed during  $[t_1, t_2]$ , each in  $O(\log N)$  time. In the final level  $\ell$ , each position in  $E_\ell[s+p_1+1, s+p_2]$  corresponds to an event where the triple  $(s, p, o)$  is successively inserted

(1) and deleted (0). Those offsets  $p_1 < p \leq p_2$  can be mapped to the corresponding time instants as we return from the recursive traversal of the VBT. Say that from our range in  $E_d[s, e]$  we went to the left child, thereby arriving at the range  $E_{d+1}[s, e - r]$  with  $r := \text{rank}(B_d, s, e)$ . Then, a position  $E_{d+1}[s - 1 + p]$  corresponds to the position  $E_d[s + \text{select}_0(B_d, s, p)]$ , where  $\text{select}_b(B_d, s, p)$  is the position of the  $p$ th occurrence of bit  $b \in \{0, 1\}$  in  $B_d[s..]$ . Analogously, if we went to the right child of  $E_d[s, e]$ , arriving at the range  $E_{d+1}[e - r + 1, e]$ , a position  $E_{d+1}[e - r + p]$  corresponds to the position  $E_d[s + \text{select}_1(B_d, s, p)]$ . Query *select* is solved in constant time using  $o(|B_d|)$  bits on top of  $B_d$  [11, 29]. After  $O(\log N)$  steps we reach the root of  $V$ , where the timestamp  $p$  can be binary searched among the  $p_l$  values assigned to the intervals  $[ts_l, te_l]$  to convert it to time instants in  $\mathcal{T}$ .

**THEOREM 7.** *Let  $G$  be a temporal graph with  $N$  tuples. Then, there is a data structure using  $O(N)$  space that can report all the occ occurrences of a single-triple-pattern query  $Q = \{(s, p, o)\}$  in the labeled graph  $G_{[t_1, t_2]}$ , for any time interval  $[t_1, t_2]$  given with  $Q$ , in time  $O((1 + \text{occ}) \log N)$ . It can also list each maximal time interval where each occurrence appears in time  $O(\log N)$ .*

Listing the time intervals where a triple pattern existed in  $[t_1, t_2]$  also permits tracking the differences between  $G_{t_1-1}$  and  $G_{t_2-1}$ , an important operation on versioned graphs. This can go from tracking a single triple  $(s, p, o)$  to listing all the changes in the graph.

## D QUERIES WITH DURATION (EXTENDED VERSION)

In some cases we are interested in requiring that the solutions to the queries last for some time. We consider two cases of such queries. A first one is to establish a time interval where the solution must always hold, so that we are interested only in that time interval. In other words, we must run the query on  $G_{[t_1, t_2]}$ ; recall Def. 7. A second one does not fix the exact times, but sets a minimum duration  $\delta$  along which the reported solutions must hold [21].

### D.1 Querying the Graph $G_{[t_1, t_2]}$

The idea used to query  $G_{[t_1, t_2]}$  in optimal time can be extended to query  $G_{[t_1, t_2]}$ :  $\text{rank}(E_d, s_v + p_1, s_v + p_2) = p_2 - p_1 + 1$  iff the node  $s_v$  exists throughout the whole period between the local offsets  $p_1$  and  $p_2$ . Therefore, we can use Algorithm 5 with the only change that we require that all the bits in the area of  $E_d$  are 1s, not just one of them. This strategy does not yield worst-case time guarantees in terms of the size of  $G_{[t_1, t_2]}$ , however: it is possible that a BT node may have existed all along  $[t_1, t_2]$ , while none of its children have. We can therefore traverse large parts of the VBT just to find out that there are no matches of  $Q$  in  $G_{[t_1, t_2]}$  (which could even be empty). On the other hand, the algorithm is still wco with respect to the size of  $G_t$  for any  $t \in [t_1, t_2]$ : if the subtree of a node  $v$  at depth  $d$  does not exist in  $G_t$ , then the corresponding bit  $E_d$  will be zero and the algorithm will not attempt to explore it. Instead of satisfying the AGM bound on  $|G_{[t_1, t_2]}| = |\cap_{t \in [t_1, t_2]} G_t|$ , we satisfy it in terms of  $\min_{t \in [t_1, t_2]} |G_t|$ . This proves the first part of Thm. 3.

### D.2 Setting a minimum duration $\delta$

Consider the problem of solving a BGP on the triples  $(s, p, o)$  of a temporal graph  $G$  so that the BGP exists in  $G$  during an interval of  $\delta$  time units or more. We solve this query on  $G$ , adding the same variable  $t$  as the fourth component of all the triple patterns, and use the LTJ tries where the time component  $\tau$  is at the end, as in join-first approaches. Note that those tries do not use the linear-space representation of Section 6. Instead, we will use a more sophisticated data structure to store the children of each LTJ trie node  $v$  in order to obtain relevant time guarantees.

Let  $u_1, \dots, u_k$  be the children of node  $v$ , with increasing values  $x_1, \dots, x_k$ . In addition to storing an array with the (values of) the nodes  $u_1, \dots, u_k$ , we store  $k$  points  $(i, \delta_i)$  in a geometric grid, where  $\delta_i$  is the longest duration of a tuple of  $G$  stored below node  $u_i$  (here we refer to the original durations of the tuples, before we mapped them to  $[0, T)$ ). When it comes to solve  $\text{leap}(v, x)$ , we look for the point  $(i, \delta_i)$  with minimum value  $i$  such that  $x_i \geq x$  and  $\delta_i \geq \delta$ . This is an orthogonal range geometric query in two dimensions, which a linear-space data structure solves in  $O(\log N)$  time [10], thereby not affecting our complexities. Note that, as we represent times in  $\mathcal{T} \setminus \mathcal{T}_G$  with their predecessors in  $\mathcal{T}_G$ , we work with the maximum possible durations and do not lose any solution.

We leave variable  $t$  to be bound at the end. When we reach the last level,  $\tau$ , in all the triple patterns of the BGP, we must intersect the  $m$  time intervals, aiming again at finding time ranges of length at least  $\delta$ . In this level we also convert every node  $[ts_i, tf_i]$  into a point  $(i, \delta_i)$  of a two-dimensional grid, with  $\delta_i$  the actual duration of the interval  $[ts_i, tf_i]$ . We can then run the intersection algorithm for the time level as described in Section 5.3 (precisely, the version with intervals of Algorithm 3), where  $\text{leap}(v, t_0)$  is solved as follows. First we find  $i$  such that (1)  $ts_i \leq t_0 < te_i$  or (2)  $te_i \leq t_0 < ts_{i+1}$ . In case (1), if the duration of  $[t_0, te_i]$  is at least  $\delta$ ,<sup>2</sup> we return  $t^* := t_0$ . In any other case, the interval  $[ts_i, te_i]$  is not useful and we search the grid for the point  $(j, \delta_j)$  with smallest coordinate  $j$  such that  $j > i$  and  $\delta_j \geq \delta$ . This yields the closest time interval to the right that contains an edge whose duration is long enough, and is found in  $O(\log N)$  time with the geometric data structure.

It is easy to see that our search algorithm works exactly as if we had run the query on  $G$ , yet completely ignoring the edges shorter than  $\delta$ . This proves the second part of Thm. 3.

<sup>2</sup>How this is defined depends on how we model durations. For example, if  $\mathcal{T} = \mathbb{N}$ , we may understand that the interval  $[2, 3]$  has a duration of 1 (if we count the number of instants it spans) or less than 1 (if we count its length on the real line).

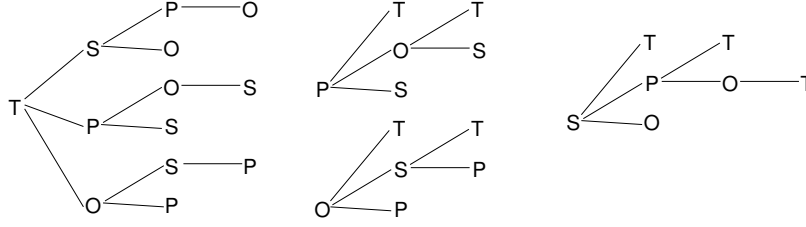


Figure 9: The meta-trie of all the LTJ tries we need to build.

## E PARTIAL TRIES AND TRIES SWITCHING (EXTENDED VERSION)

Although our data structure uses linear space, the constant is high in practice because of the need to store  $4! = 24$  tries with all the possible orderings of  $\{s, p, o, t\}$ . Since each trie has 4 levels and all the tuples are mentioned once in every level before the time level and twice since the time level, a (slightly pessimistic) upper bound on the number of LTJ trie nodes stored is  $156N$  for a temporal graph of  $N$  tuples. Further, our representation for the trie levels requires two words per node (i.e., two events), and versioned tries after the time level require four words per node (i.e., each event induces  $\ell$  in each bitvector  $B_d$  and  $E_d$ ), which sets the count to  $228N$  words. The space for the trie pointers can in general be dismissed by using compact topology representations [23]. Considering that we need  $5N$  words to represent the  $N$  tuples  $(s, p, o, [t_i, t_f])$  in raw form, the blowup factor in the space is 45.6.

In order to reduce this overhead we make use of *trie switching* [6] and *partial tries*. Another space-saving device is the possibility of sharing trie prefixes. For example, the tries for  $sPO$  and  $sPTO$  can share their first two levels; the nodes of the level  $sp$  concatenate two sequence of children: those of the trie that continues with  $o$  and those of the trie that continues with  $to$ . This can also be emulated with the succinct topology representations that do not use pointers [23], because we can simulate that they have a first child continuing with  $o$  and a second one with  $to$ . Our linear-space representation of Section 6 can be similarly adapted. Those fake pointers add up to  $O(N)$  bits of space.

With these mechanisms, we can define a *meta-trie* that stores all the paths that are stored in the index; each path corresponds to a (possibly partial) LTJ trie. The shared paths in the meta-trie correspond to shared paths in the LTJ tries. Therefore, the number of nodes in the meta-trie correspond to the factor multiplying  $N$  in our storage space, if multiplied by the appropriate factor: 1 for levels before time, 2 for the time level, 4 for the levels after time. Figure 9 contains a meta-trie that suffices to run LTJ on any instantiation order, and which uses just  $76N$ , 15.2 times the space needed to store the raw data.

## F GENERATION OF TBGPS

We filter disconnected and duplicate BGPs (modulo graph isomorphism). In order to ensure that the BGPs we use for experiments are likely to touch the non-trivial temporal annotations, we define a score to prioritize BGPs with more triple patterns whose predicates have a higher ratio of triples with non-trivial temporal annotations. More specifically, for each predicate  $p_i$ , in case it is a constant, we define  $r_i$  as the ratio of triples with predicate  $p_i$  that have non-trivial temporal annotations in the graph; otherwise, if  $p_i$  is a variable, we define  $r_i$  as 0. For each BGP extracted from the log of the form  $Q = \{(s_1, p_1, o_1), \dots, (s_m, p_m, o_m)\}$ , we compute the score  $\sum_{i=1}^m r_i$ . We then extract the top 1,000 BGPs from the query logs per this score. Further filtering BGPs with constants not appearing in the graph, we arrive at 972 queries. Finally, we extend each triple pattern with a shared temporal variable  $v \in \mathcal{V}_t$  to generate  $Q' = \{(s_1, p_1, o_1, v), \dots, (s_m, p_m, o_m, v)\}$ .